

INVENTORY MANAGEMENT APPLICATION



Submitted in partial fulfillment of the Degree of

Bachelor of Technology

in

Computer Science Engineering

by

Palak Patodi (2316496)

Somya Jain (2316571)

Soumya Arora (2316572)

Suhani Jain (2316576)

Vanshree (2316602)

Under the supervision of

Dr. Mainaz Faridi

Associate Professor

DEPARTMENT OF COMPUTER SCIENCE

BANASTHALI VIDYAPITH, RAJASTHAN

2025–26

Certificate

This is to certify that the project work titled "**Inventory Management Application**" submitted by **Palak Patodi, Somya Jain, Soumya Arora, Suhani Jain, and Vanshree** in partial fulfillment for the award of degree of B. Tech in CSE of Banasthali Vidyapith has been carried out under my supervision. This work has not been submitted partially or wholly to any other University or Institute for the award of this or any other degree or diploma.


Dr. Mainaz Faridi

Designation: Associate Professor

Place: Banasthali Vidyapith

Date: 08-04-2026

Abstract

The Inventory Management Application (IMA) is a client server application developed for the Faculty of Mathematics and Computing at Banasthali Vidyapith. The central motivation is to replace the existing manual, paper-based inventory tracking system with a fully digital, secure, and role-based solution that enhances operational efficiency, transparency, and accountability across all inventory-related activities.

The system manages departmental inventory across four primary categories: stationery, furniture, computer sets, and miscellaneous items. Three distinct user roles are supported — Inventory Manager, Head of Department (HOD), and Dean — each with clearly defined access permissions and functional capabilities. The Inventory Manager handles all day-to-day operational tasks including adding, modifying, viewing, and searching product records, recording issuances to faculty members, processing returns, and managing supplier and bill information. The HOD and Dean exercise administrative oversight through budget analysis reports, and multi-level purchase request approval workflows.

Key features include secure role-based authentication with account lockout after three failed login attempts, complete CRUD operations on product and supplier records, automated low-stock alert notifications, dynamic report generation on weekly, monthly, and yearly schedules, budget analysis dashboards, faculty usage reports, and a structured multi-level procurement approval workflow. All reports can be exported in PDF format for record-keeping and audit purposes.

The application is built using JAVA 25 LTS, JavaFX for the graphical user interface, and MySQL 8.0 as the backend relational database, accessed via the JDBC API. The system follows a three-tier architecture separating presentation, business logic, and data concerns. The database is normalised to the Third Normal Form (3NF) to ensure data integrity and minimise redundancy. The application is designed to run on standard departmental hardware under Windows 10 or higher.

Acknowledgement

We take this opportunity to express our sincere gratitude and deep regards to our project mentor, **Dr. Mainaz Faridi**, Associate Professor, Department of Computer Science, Banasthali Vidyapith, for her exemplary guidance, valuable feedback, consistent monitoring, and constant encouragement throughout the course of this project. Her expertise and constructive suggestions were pivotal in shaping the direction and quality of this work.

We are grateful to the **Head of the Department of Computer Science**, Banasthali Vidyapith, for providing the necessary academic resources, infrastructure, and a conducive environment for project development. We also thank all the faculty members of the Department of Computer Science for their support throughout the academic year.

We extend our gratitude to the **Faculty of Mathematics and Computing** for sharing insights about the inventory challenges that inspired this project. Their real-world perspective ensured that the solution addresses genuine institutional needs rather than hypothetical scenarios. The staff members who participated in the requirements-gathering sessions gave us invaluable practical information about the pain points of the existing manual system.

We also wish to acknowledge the open-source community whose tools and frameworks — Java SE, JavaFX, MySQL, Apache Maven, and NetBeans IDE — made this project technically achievable within the constraints of an academic timeline. The documentation and community forums associated with these technologies served as an essential reference throughout development.

Finally, we express our deepest thanks to our families and peers whose constant encouragement and moral support enabled us to remain focused and motivated throughout the duration of this project.

Palak Patodi, Somya Jain, Soumya Arora, Suhani Jain, Vanshree

Department of Computer Science, Banasthali Vidyapith, Rajasthan, 2025–26

Contents

1	Introduction	1
1.1	Overview	1
1.2	Background	1
1.3	Problem Statement	2
1.4	Objectives	3
1.5	Scope of the Project	3
1.6	Technology Stack	4
1.7	Report Organisation	4
2	Requirement Analysis	5
2.1	Requirement Specification	5
2.1.1	Functional Requirements	5
2.1.2	Non-Functional Requirements	7
	Performance Requirements	7
	Security Requirements	7
	Reliability and Data Integrity	8
	Usability	8
	Portability	8
2.2	Hardware and Software Requirements	8
2.2.1	Hardware Requirements	8
2.2.2	Software Requirements	9
2.3	Feasibility Study	9
2.3.1	Technical Feasibility	9
2.3.2	Operational Feasibility	9
2.3.3	Economic Feasibility	10
2.4	Product Functions	10
2.5	Use-Case Diagrams	11
2.5.1	Actors	11
2.5.2	Use-Case Diagram	11
2.5.3	Use-Case Descriptions	11
2.5.4	Use-Case Templates	13
3	System Design	20
3.1	High-Level Design	20
3.2	Package Structure	20
3.3	ER Diagram	21
3.4	Database Design	22
3.5	Data Flow Diagrams	26

3.5.1	DFD Level 0 — Context Diagram	26
3.5.2	DFD Level 1 — Inventory Manager	27
3.5.3	DFD Level 1 — HOD and Dean	29
3.6	Sequence Diagrams	29
4	Coding	31
4.1	Module 1: Notification Service (NotificationService.java)	31
4.2	Module 2: Login and Authentication (LoginUI.java)	32
4.3	Module 3: Add Inventory / Bill Receipt (AddInventoryUI.java)	34
4.4	Module 4: Issue Inventory (IssueUI.java)	36
4.5	Module 5: Return Inventory (ReturnUI.java)	37
4.6	Module 9: Report Generation (ReportUI.java)	39
4.7	Module 10: Budget Analysis (BudgetAnalysisUI.java)	41
4.8	Module 7: Add Order (AddOrderUI.java)	44
4.9	Module 8: Manage Inventory (ManageInventoryUI.java — Key Sections)	46
4.10	Module 6: Dashboard (DashboardUI.java — Key Sections)	49
5	Testing	51
5.1	Testing Strategy	51
5.2	Unit Testing	51
5.3	Integration Testing	54
5.4	System Testing	55
5.5	Stress Testing	56
5.6	Acceptance Testing	56
6	User Interfaces	57
6.1	Login Screen	57
6.2	Main Dashboard	58
6.3	Update	59
6.4	Issue Inventory Screen	60
6.5	Return Screen (Faculty to Inventory)	60
6.6	Return to Supplier (RTS) Screen	61
6.7	View	62
6.8	Budget Analysis Screen	63
6.9	Reports	63
6.10	Inventory Request	65
A	Glossary	66
	References	68

List of Figures

2.1	Use-Case Diagram for Inventory Management Application (IMA)	11
2.2	Use-Case Template — UC-01: Login	13
2.3	Use-Case Template — UC-02: Retrieve Password	14
2.4	Use-Case Template — UC-03: Bill Recieved	14
2.5	Use-Case Template — UC-04: Modify Product Details	15
2.6	Use-Case Template — UC-05: Order Placed	15
2.7	Use-Case Template — UC-05: Issue	16
2.8	Use-Case Template — UC07: Issue	16
2.9	Use-Case Template — UC08	17
2.10	Use-Case Template — UC09: Generate Budget Analysis Report	17
2.11	Use-Case Template — UC10: View Report	18
2.12	Use-Case Template — UC11: View User Manual	18
2.13	Use-Case Template — UC12: Request Inventory Purchase	19
2.14	Use-Case Template — UC13: Logout	19
3.1	Entity-Relationship Diagram for IMA Database	22
3.2	DFD Level 0 — Context Diagram for IMA	27
3.3	DFD Level 1 — Inventory Manager Processes	28
3.4	DFD Level 1 — HOD and Dean Processes	29
3.5	Update Button — IMA	30
6.1	Login Screen — Inventory Management Application	57
6.2	Retrieve Password Screen — Inventory Management Application	58
6.3	Main Dashboard — Inventory Management Application	58
6.4	Bill Recieved form — IMA	59
6.5	Add Order Form— IMA	59
6.6	Modify — IMA	59
6.7	Issue Inventory Form — IMA	60
6.8	Return Button — IMA	60
6.9	View — IMA	62
6.10	Order Details— IMA	62
6.11	Report — IMA	63
6.12	Weekly Report — IMA	64

6.13 Inventory Request Workflow of the Inventory Management System . 65

List of Tables

1.1	Technology Stack Used in IMA	4
2.1	Hardware Requirements for IMA	8
2.2	Software Requirements for IMA	9
2.3	Use-Case Descriptions	12
3.1	Package Structure of IMA	21
3.2	User Table Schema	22
3.3	Role Table Schema	23
3.4	Product Table Schema	23
3.5	Product_Type Table Schema	23
3.6	Supplier Table Schema	24
3.7	Order Table Schema	24
3.8	Bill Invoice Table Schema	25
3.9	Issue Table Schema	25
3.10	Return_Table Schema	26
3.11	RTS_Table Schema	26
5.1	Unit Test Cases	51
A.1	Glossary of Terms	66

Chapter 1

Introduction

1.1 Overview

Inventory management is one of the most fundamental administrative functions in any academic institution. Colleges and universities maintain a wide variety of physical assets — from stationery and office supplies to furniture, computer equipment, and miscellaneous items — all of which must be systematically tracked, allocated, and monitored to ensure smooth departmental operations. Without an organised system, institutions face frequent stock shortages, misallocation of resources, loss of records, and difficulties during audits.

The Faculty of Mathematics and Computing at Banasthali Vidyapith has historically managed its inventory through manual registers, physical files, and spreadsheets. While this approach may have been sufficient for smaller inventories, the growing scale of departmental assets — including an expanding computer laboratory, increasing stationery requirements, and diverse furniture needs — has rendered the manual system increasingly inadequate and error-prone.

This project presents the **Inventory Management Application (IMA)**, a client server application specifically designed to address these challenges. The IMA digitises the complete inventory lifecycle — from procurement and storage to issuance, return, and reporting — within a secure, role-based environment accessible only to authorised personnel. The system was conceived, designed, and implemented entirely by students of the Department of Computer Science, and its source code is publicly hosted at

1.2 Background

The concept of computerised inventory management is well established in commercial and industrial settings. Enterprise Resource Planning (ERP) systems, warehouse management tools, and point-of-sale software have long used digital inventory tracking to improve efficiency and reduce human error. However, institutional environments — particularly academic departments — often lack the resources or justification to implement large-scale ERP systems. What they require is a lightweight, purpose-built application tailored to their specific workflows and constraints.

The Faculty of Mathematics and Computing currently tracks inventory man-

ually. Office staff maintain separate physical registers for stationery, furniture, computer sets, and other items. Purchase requests are submitted through informal channels, and the issuance of items to faculty members is recorded on paper. This approach has several well-documented drawbacks: difficulty in tracking real-time stock levels, inability to generate instant reports, absence of a structured approval workflow, and vulnerability to data loss or corruption.

A comprehensive survey conducted among the office staff of the faculty revealed that approximately 4–6 hours per week are spent compiling inventory information from paper registers for routine management reviews. Furthermore, 3 out of 5 stockout incidents in the past academic year were attributed to delayed awareness of low stock levels, which a digital alert system could have prevented entirely. These findings provided the quantitative basis for the IMA project.

1.3 Problem Statement

The Faculty of Mathematics and Computing relies entirely on manual, paper-based methods to manage its departmental inventory. This approach results in the following critical problems:

1. **Lack of Real-Time Visibility:** Staff cannot determine current stock levels without physically counting items or searching through paper registers, resulting in unexpected stockouts or unnecessary over-purchasing.
2. **No Structured Approval Workflow:** Purchase requests are submitted informally with no standardised process for submission, review, and approval, leading to delays and occasional unauthorised purchases.
3. **Inability to Generate Reports:** Producing a weekly or monthly usage report requires hours of manual data compilation, making routine management reviews impractical.
4. **Risk of Data Loss:** Physical registers are susceptible to damage, misplacement, or destruction with no backup mechanism, meaning historical data could be permanently lost.
5. **Absence of Role-Based Access:** Any staff member with physical access to the registers can read or modify records without restriction, creating a security and accountability risk.
6. **Inefficient Issuance and Return Tracking:** Recording item issuances and tracking returns is cumbersome and error-prone, often resulting in unaccounted assets and discrepancies during annual audits.

7. **No Alert Mechanism:** There is no system to automatically notify staff when stock falls below a safe minimum level, leading to last-minute procurement crises that disrupt departmental operations.

1.4 Objectives

The primary objectives of the Inventory Management Application are:

1. To develop a secure, role-based desktop application that automates departmental inventory management for the Faculty of Mathematics and Computing.
2. To provide a unified digital platform for tracking stationery, furniture, computer sets, and miscellaneous inventory across their complete lifecycle from procurement to disposal.
3. To implement a multi-level purchase request approval workflow (Inventory Manager → HOD → Dean) ensuring proper authorisation before procurement.
4. To automate the generation of inventory reports on weekly, monthly, and yearly bases for Inventory Managers, enabling data-driven management decisions.
5. To provide HOD and Dean with budget analysis reports for strategic resource allocation and decision-making.
6. To implement an automated low-stock alert system triggered when any product's quantity falls below a predefined minimum threshold, eliminating reactive stockout situations.
7. To maintain complete, auditable records of all inventory transactions for accountability, traceability, and compliance with institutional audit requirements.
8. To design the system for portability on standard departmental PCs running Windows 10 or higher with JAVA 25 LTS.

1.5 Scope of the Project

The IMA is scoped exclusively for the Faculty of Mathematics and Computing at Banasthali Vidyapith. It manages four inventory categories: stationery, furniture, computer sets, and miscellaneous items. It supports three user roles: Inventory Manager, HOD, and Dean. The system handles the complete inventory lifecycle

including procurement order management, bill receipt, stock tracking, issuance to faculty, returns from faculty, returns to supplier (RTS), and report generation.

The system is explicitly **out of scope** for integration with external enterprise tools, web-based access from outside the local network, financial accounting and payroll functions, or deployment across other faculty departments without configuration changes. Integration with external email services is provided for purchase request notifications, but the application itself does not function as an email client.

1.6 Technology Stack

Table 1.1: Technology Stack Used in IMA

Component	Technology
Programming Language	JAVA 25 LTS
GUI Framework	JavaFX
Database	MySQL 8.0
Database Connectivity	JDBC (Java Database Connectivity API 4.3)
Build Tool	Apache Maven
IDE	NetBeans
Operating System	Windows 10 or higher
Email API	BBC Mail (for purchase request notifications)

1.7 Report Organisation

Chapter 2 presents the Requirement Analysis (SRS) including functional and non-functional requirements, hardware/software requirements, feasibility study, product functions, and use-case diagrams. Chapter 3 covers System Design (SDS) including three-tier architecture, ER diagram, database design, and DFDs. Chapter 4 presents the main coding modules with actual source code from the GitHub repository. Chapter 5 describes the testing strategy, test cases, and results. Chapter 6 presents the User Interface screens. References and Appendix follow.

Chapter 2

Requirement Analysis

2.1 Requirement Specification

This chapter presents the Software Requirements Specification (SRS) for the IMA, developed in accordance with IEEE Std 830-1998. The SRS defines all functional and non-functional requirements identified through stakeholder consultation with the Faculty of Mathematics and Computing's office staff, HOD, and Dean. Requirements were gathered through a combination of structured interviews, direct observation of existing manual workflows, and analysis of specimen paper registers used by the department.

2.1.1 Functional Requirements

Authentication and Access Control:

- REQ-F01: The system shall require all users to authenticate with a valid username and password before accessing any functionality.
- REQ-F02: The system shall support three user roles: Inventory Manager, HOD, and Dean, each with a role-specific interface and feature set.
- REQ-F03: The system shall lock a user account after three consecutive unsuccessful login attempts and display a warning message.
- REQ-F04: The system shall provide a password retrieval mechanism using a secret security question, implemented as a three-step flow.
- REQ-F05: Authenticated users shall be able to change their own password from within the application.

Product Management:

- REQ-F06: The Inventory Manager shall be able to add new product records including name, description, category, quantity, and minimum threshold.
- REQ-F07: The Inventory Manager shall be able to modify existing product records through the Manage Inventory module.
- REQ-F08: Product records shall support soft-delete using an ACTIVE/IN-ACTIVE record status flag to preserve audit history.

- REQ-F09: All authenticated users shall be able to view product details.
- REQ-F10: Users shall be able to search and filter products by name, code, or availability status.

Order and Bill Management:

- REQ-F11: The system shall track purchase orders via an order table with statuses: IN_PROCESS and PAID.
- REQ-F12: The Inventory Manager shall record received bill details including bill number, amount, received date, and quantity received against a specific order.
- REQ-F13: The system shall validate that the quantity received against a bill does not exceed the remaining order balance.
- REQ-F14: Bill status shall automatically update to COMPLETE and order status to PAID when total received quantity equals the ordered quantity.

Issuance and Returns:

- REQ-F15: The Inventory Manager shall record item issuances to faculty members or departments with complete details including recipient, department, quantity, reason, and date.
- REQ-F16: The system shall prevent issuance of a quantity exceeding current available stock, with a clear error message indicating available stock.
- REQ-F17: The system shall use atomic database transactions for issuance operations to maintain data integrity and consistency.
- REQ-F18: The Inventory Manager shall record item returns from faculty to inventory, automatically updating stock levels within an atomic transaction.
- REQ-F19: The system shall support Return to Supplier (RTS) operations for returning defective or surplus items to vendors.

Purchase Request Workflow:

- REQ-F20: The Inventory Manager shall submit purchase requests for new items or stock replenishment via an email-based notification system.
- REQ-F21: Submitted requests shall be routed to the HOD for first-level review and approval.

- REQ-F22: For computer sets or large orders, the Dean shall provide additional approval.
- REQ-F23: The system shall send email notifications to the HOD and Dean when a request is submitted or forwarded.

Reporting and Analytics:

- REQ-F24: The system shall generate weekly, monthly, and yearly inventory reports for Inventory Managers.
- REQ-F25: HOD and Dean shall have access to a Budget Analysis Report (with pie chart and bar chart visualisations) and a Faculty Usage Report.
- REQ-F26: All reports shall be exportable in PDF format.

View and Alert Functions:

- REQ-F27: The system shall provide table views for Order Details, Issue and Return Details, and RTS Details from the dashboard.
- REQ-F28: The system shall automatically trigger an alert when any product's stock falls at or below the minimum quantity threshold.
- REQ-F29: In-app notifications shall be accessible from the user profile icon in the dashboard.

2.1.2 Non-Functional Requirements

Performance Requirements

The application shall load the main dashboard within 3 seconds on standard departmental hardware. Database queries shall complete within 2 seconds for datasets up to 10,000 records. Concurrent access by up to 3 Inventory Managers, 1 HOD, and 1 Dean shall not cause performance degradation or database deadlocks.

Security Requirements

Role-based access control shall be enforced consistently at the feature level. HOD and Dean users shall not have access to the Update, Issue, Return, or Request Inventory functions. Manager users shall not have access to the Budget Analysis function. The system shall lock accounts after three consecutive failed login attempts.

Reliability and Data Integrity

All stock-modifying operations (bill receipt, issuance, return, RTS) shall execute as atomic database transactions using JDBC transaction management. In the event of failure, the system shall roll back all changes to maintain consistency. Soft-delete mechanisms shall preserve all historical records for audit purposes.

Usability

The GUI shall provide clear navigation menus, inline validation error messages, and an in-app PDF User Manual. The warm brown-beige colour scheme (#E2C49F, #8B5E34) shall be maintained consistently across all screens. Keyboard shortcuts shall be provided for common operations where applicable.

Portability

The application shall run on Windows 10 or higher and Linux distributions supporting JAVA 25 LTS and MySQL 8.0. The Maven build system shall ensure all dependencies are managed automatically without manual JAR installation.

2.2 Hardware and Software Requirements

2.2.1 Hardware Requirements

Table 2.1: Hardware Requirements for IMA

Component	Minimum Specification
Processor	Intel Core i3 or equivalent, 2.0 GHz or higher
RAM	Minimum 4 GB
Storage	Minimum 500 MB free disk space
Display	1024×768 resolution or higher
Input Devices	Standard keyboard and mouse
Network	LAN or localhost for database connectivity; internet for email notifications

2.2.2 Software Requirements

Table 2.2: Software Requirements for IMA

Component	Specification
Operating System	Windows 10 or higher
Java Runtime	JAVA 25 LTS (JRE 17)
GUI Framework	JavaFX (via Maven dependency)
Database Server	MySQL 8.0
Database Driver	MySQL Connector/J (JDBC)
Build Tool	Apache Maven 3.6+
Email Library	Jakarta Mail API
IDE (Development)	NetBeans IDE
Version Control	Git / GitHub

2.3 Feasibility Study

2.3.1 Technical Feasibility

The IMA is built on well-established and freely available technologies. JAVA 25 LTS is an actively maintained Long-Term Support (LTS) release supported until at least 2029. JavaFX provides a modern cross-platform GUI framework with rich UI controls including TableView, ComboBox, DatePicker, and Chart components. MySQL is among the world's most widely used relational databases and is freely available under the GPL licence. Standard departmental PCs meet all hardware requirements. All required tools are open-source with zero licensing costs. The project team members have the requisite technical skills in Java, SQL, and JavaFX from prior coursework. The project is therefore fully technically feasible.

2.3.2 Operational Feasibility

The end users — office staff acting as Inventory Managers, and administrative users (HOD and Dean) — are computer-literate professionals accustomed to digital tools such as spreadsheets and email. The JavaFX-based GUI is designed to be intuitive, with contextual dropdowns, inline error messages, and a bundled User Manual in PDF format accessible directly from the dashboard. The system directly addresses existing operational pain points identified through stakeholder interviews, ensuring high user acceptance. The project is operationally feasible.

2.3.3 Economic Feasibility

All technologies used are available under free or open-source licences, resulting in zero software cost. Development is carried out by the student team as an academic project, eliminating development labour costs. Existing departmental hardware meets all requirements without additional investment. The system will significantly reduce the approximately 4–6 hours per week currently spent on manual record-keeping. The project is economically feasible with a clear return on investment within a single academic semester.

2.4 Product Functions

The IMA provides the following high-level product functions:

1. **Authentication and Access Control:** Secure login for all three roles with role-specific interface, three-step password recovery, and account lockout protection.
2. **Product Management:** View and search the product catalog organised by four inventory categories, with soft-delete support for maintaining audit history.
3. **Order and Bill Management:** Tracking of purchase orders placed with suppliers and received bills with automatic status transitions from IN_PROCESS to PAID.
4. **Issuance and Return Tracking:** Atomic transactional recording of item issuances to faculty and returns to inventory, with automatic stock updates.
5. **Return to Supplier (RTS):** Atomic recording of items sent back to suppliers for defects or excess stock, with corresponding stock adjustment.
6. **Multi-Level Purchase Request Workflow:** Electronic routing of purchase requests through email notifications to HOD and Dean.
7. **Automated Reporting:** Weekly, monthly, and yearly inventory summaries with budget analysis and faculty usage reports exportable as PDF.
8. **Low-Stock Alerts and In-App Notifications:** Automatic notifications when any product's stock meets or falls below the minimum threshold.
9. **Data View Tables:** Real-time table views for Order Details, Issue and Return Details, and RTS Details drawn live from the MySQL database.

2.5 Use-Case Diagrams

2.5.1 Actors

- **Inventory Manager:** Primary operational user; 2–3 office staff members responsible for all day-to-day inventory activities including adding orders, receiving bills, issuing items, processing returns, and generating reports.
- **HOD:** Reviews budget and faculty usage reports, provides first-level approval for purchase requests.
- **Dean:** Provides final approval for major or computer-related purchases and accesses strategic reports.

2.5.2 Use-Case Diagram

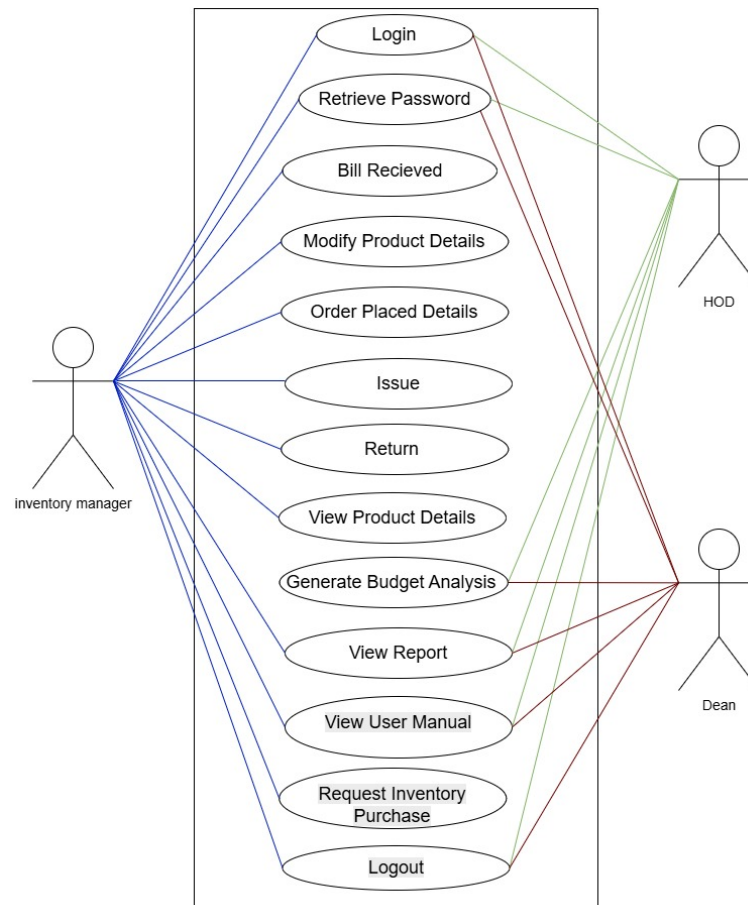


Figure 2.1: Use-Case Diagram for Inventory Management Application (IMA)

2.5.3 Use-Case Descriptions

Table 2.3: Use-Case Descriptions

UC	Use Case	Actor	Pre-condition	Post-condition
UC01	Login	All	Registered account exists	Authenticated; role-specific dashboard shown
UC02	Retrieve Password	All	User on login screen	Password reset successfully via secret question
UC03	Place Order	Manager	Logged in as Manager	Order saved in order_table
UC04	Add Bill / Receive Inventory	Manager	Active IN_PROCESS order exists	Bill saved; stock incremented; statuses updated
UC05	View Order Details	Manager	Order records exist	Order details displayed in table view
UC06	Issue Inventory	Manager	Product with sufficient stock exists	Issue recorded; stock decremented atomically
UC07	Return (Faculty to Inventory)	Manager	Issued records with pending returns exist	Return recorded; stock incremented
UC08	Return to Supplier (RTS)	Manager	Bill records exist for product	RTS record saved; stock adjusted
UC09	View Issue & Return Details	Manager	Issue and return records exist	Combined table displayed
UC10	View RTS Details	Manager	RTS records exist	RTS table displayed
UC11	View Weekly Report	Manager	Transaction data exists	Weekly report generated and displayed
UC12	View Monthly Report	Manager	Transaction data exists	Monthly report generated and displayed

UC	Use Case	Actor	Pre-condition	Post-condition
UC13	View Yearly Report	Manager	Transaction data exists	Yearly report generated and displayed
UC14	Budget Analysis Report	HOD, Dean	Transaction data exists	Pie and bar chart report displayed
UC15	Request Inventory Purchase	Manager	Logged in; recipient selected	Email sent to HOD or Dean
UC16	Low Stock Alert	Manager	Stock $\leq$ minimum threshold	Alert dialog displayed; notification saved
UC17	View Notifications	All	Notifications exist in service	Notification panel displayed
UC18	View User Manual	All	PDF manual in resources	PDF opened in default viewer
UC19	Logout	All	User is logged in	Session terminated; login screen shown

2.5.4 Use-Case Templates

The following pages provide placeholders for the 13 use-case templates. Replace each placeholder box with your diagram image using the `\includegraphics` command shown inside each box.

UC-01: Login	
Uname	Login
Uno.	01
Actor	Inventory Manager, HOD, Dean
Pre-condition	User must have a registered account.
Post-condition	User is authenticated and logged in. System is ready for subsequent use cases.
Normal flow	User enters valid credentials, system authenticates and then user is redirected to the main interface where available tasks are displayed.
Abnormal flow	Invalid credentials shows an error and allow retries. User can select "forgot password" to reset credentials.

Figure 2.2: Use-Case Template — UC-01: Login

UC-02: Retrieve Password	
Uname	Retrieve Password
Uno.	02
Actor	Inventory Manager, HOD, Dean
Pre-condition	User is on the login screen. User must have a registered account.
Post-condition	A password reset question will be sent to the user. User can create a new password
Normal flow	User clicks "Retrieve password", provides identification and receives a question change their password
Abnormal flow	Invalid user details show an error message is displayed.

Figure 2.3: Use-Case Template — UC-02: Retrieve Password

UC-03: Bill Recieved	
Uname	Bill Recieved
Uno.	03
Actor	Inventory Manager
Pre-condition	User is logged in as Inventory Manager. "Bill Recieved" form is accessible.
Post-condition	The new bill is saved in the system.
Normal flow	User opens the "Bill Received" form and add the details of the form in the system.
Abnormal flow	Missing or invalid input shows an error; duplicate bill shows a warning; system failure during save displays an error message.

Figure 2.4: Use-Case Template — UC-03: Bill Recieved

UC-04: Modify Product Details

Uname	Modify Product Details
Uno.	04
Actor	Inventory Manager
Pre-condition	User is logged in as Inventory Manager.
Post-condition	Updated product details are saved, reflected and deleted in the inventory records.
Normal flow	User add and edit a new product and details of supplier and delete the bill and order details if needed.
Abnormal flow	Invalid inputs show an error; if the delete and add new option is not shown a message is displayed; save failure due to system error shows an error message.

Figure 2.5: Use-Case Template — UC-04: Modify Product Details

UC-05: Order Placed

Uname	Order Placed
Uno.	05
Actor	Inventory Manager
Pre-condition	User is logged in as Inventory Manager.
Post-condition	Order is placed in the inventory records.
Normal flow	User selects a product from the inventory list, and place a order to be added in inventory records.
Abnormal flow	If product doesn't exist, an error is shown; deletion is canceled if user declines confirmation; system error during deletion shows a failure message.

Figure 2.6: Use-Case Template — UC-05: Order Placed

UC-05: Issue Inventory	
Uname	Issue
Uno.	06
Actor	Inventory Manager
Pre-condition	User is logged in, and the product should be in inventory sytem.
Post-condition	The product will be issued to the person by the inventory manager
Normal flow	User will be able to issue the product with the required quantity.
Abnormal flow	The issue will not be able to done because the product is not in inventory system.

Figure 2.7: Use-Case Template — UC-05: Issue

UC07 — Issue	
Uname	Issue
Uno.	06
Actor	Inventory Manager
Pre-condition	User is logged in, and the product should be in inventory sytem.
Post-condition	The product will be issued to the person by the inventory manager
Normal flow	User will be able to issue the product with the required quantity.
Abnormal flow	The issue will not be able to done because the product is not in inventory system.

Figure 2.8: Use-Case Template — UC07: Issue

UC08 — Return	
Uname	Return
Uno.	07
Actor	Inventory Manager
Pre-condition	User is logged in, Product should be issued.
Post-condition	If the product is issued or return from supplier will be added to inventory and if the return to supplier, then removed from inventory.
Normal flow	User will fill the form if return is done from or to the supplier and for return from issued person then only the number of returns to be updated.
Abnormal flow	If the return is not done and is pending for a long time.

Figure 2.9: Use-Case Template — UC08

UC09 — Generate Budget Analysis Report	
Uname	Generate Budget Analysis Report
Uno.	09
Actor	HOD, Dean
Pre-condition	User is logged in, Budget and transaction data exists in the system.
Post-condition	Budget report is generated and displayed or exported in the selected format.
Normal flow	User selects "Generate Budget Analysis Report", system generates the report for viewing or jpg export.
Abnormal flow	If no data is available for the selected criteria, a "No data found" message is shown; system errors during generation display an error message.

Figure 2.10: Use-Case Template — UC09: Generate Budget Analysis Report

UC10 — View Report

Uname	View Report
Uno.	10
Actor	Inventory Manager, HOD, Dean
Pre-condition	User is logged in. Report data exists in the system.
Post-condition	Selected report is displayed on screen or made available for jpg export.
Normal flow	User navigates to the reports section, selects a report type, and the system displays the requested report.
Abnormal flow	User navigates to the reports section, selects a report type, and the system displays the requested report.

Figure 2.11: Use-Case Template — UC10: View Report

UC11 — View User Manual

Uname	View User Manual
Uno.	11
Actor	Inventory Manager, HOD, Dean
Pre-condition	User is logged in. User manual is available in the system.
Post-condition	User manual is displayed for viewing.
Normal flow	User clicks on "User Manual", and the system opens the manual in a viewer.
Abnormal flow	If the manual file is missing or corrupted, an error message is shown; system error prevents access and displays a failure message.

Figure 2.12: Use-Case Template — UC11: View User Manual

UC12 — Request Inventory Purchase

Uname	Request Inventory Purchase
Uno.	12
Actor	Inventory Manager
Pre-condition	User is logged in as Inventory Manager. Access to the purchase request form is available.
Post-condition	Purchase request is submitted and sent to HOD or Dean via email.
Normal flow	User fills out the purchase request form with item details and submits it; system saves the request and notifies the HOD or Dean for approval via email.
Abnormal flow	If required fields are missing or invalid, an error is shown; if system fails to submit the request, an error message is displayed.

Figure 2.13: Use-Case Template — UC12: Request Inventory Purchase

UC13 — Logout

Uname	Logout
Uno.	13
Actor	Inventory Manager, HOD, Dean
Pre-condition	User is logged in .
Post-condition	User session is terminated and user is redirected to the login screen.
Normal flow	User selects the logout option; system terminates the session and redirects to the login screen
Abnormal flow	If logout fails due to system error, an error message is displayed, and the user remains logged in

Figure 2.14: Use-Case Template — UC13: Logout

Chapter 3

System Design

3.1 High-Level Design

The Inventory Management Application follows a **three-tier architecture** separating the application into three logical layers:

- **Presentation Layer (JavaFX GUI):** All user interface components including login screens, dashboards, product management forms, report viewers, table views, and alert dialogs. Input validation is performed at this layer before data is passed onwards. Each UI screen is implemented as a separate Java class in the `ui` package that constructs and manages its `JavaFX Scene`.
- **Business Logic Layer (Java Classes):** Core application logic handling authentication, order and bill processing, issuance, return, RTS, report generation, purchase request email routing, and alert notifications. Key service classes include `NotificationService`, `BudgetService`, and `ReportData`. The main application controller class `Inventory` manages scene transitions and maintains global session state (logged-in user and role).
- **Data Layer (MySQL via JDBC):** All interactions with the MySQL database through the `DBConnection` utility class. JDBC executes SQL queries, prepared statements, and atomic transactions via the standard `java.sql` API. All persistence and retrieval occur here, with connection management handled per-operation for simplicity.

3.2 Package Structure

The IMA codebase is organised into the following Java packages:

Table 3.1: Package Structure of IMA

Package	Contents and Purpose
com.mycompany.inventory	Main application class <code>Inventory.java</code> — JavaFX Application entry point; manages scene transitions and session state
ui	All JavaFX screen classes: <code>LoginUI</code> , <code>DashboardUI</code> , <code>AddInventoryUI</code> , <code>AddOrderUI</code> , <code>IssueUI</code> , <code>ReturnUI</code> , <code>ManageInventoryUI</code> , <code>ReportUI</code> , <code>BudgetAnalysisUI</code>
db	<code>DBConnection.java</code> — singleton utility providing MySQL JDBC connections
model	Data model classes: <code>Product</code> , <code>Bill</code> , <code>Supplier</code> , <code>IssueModel</code> , <code>IssueRow</code> , <code>OrderRow</code> , <code>ProductRow</code> , <code>ReportData</code> , <code>BudgetData</code>
service	<code>NotificationService.java</code> — in-memory observable notification list; <code>BudgetService.java</code> — budget data retrieval

3.3 ER Diagram

The Entity-Relationship diagram models the complete database of the IMA. The central entity is **Product**, linked to **Supplier** and categorised via **Product.Type**. Purchase transactions are captured in the **Order** and **Bill Invoice** entities. Internal movements are tracked through the **Issue** entity, returned items through the **Return Table** entity, and supplier returns through **RTS Table**. The **User** entity manages authentication and role assignment via the **Role** entity.

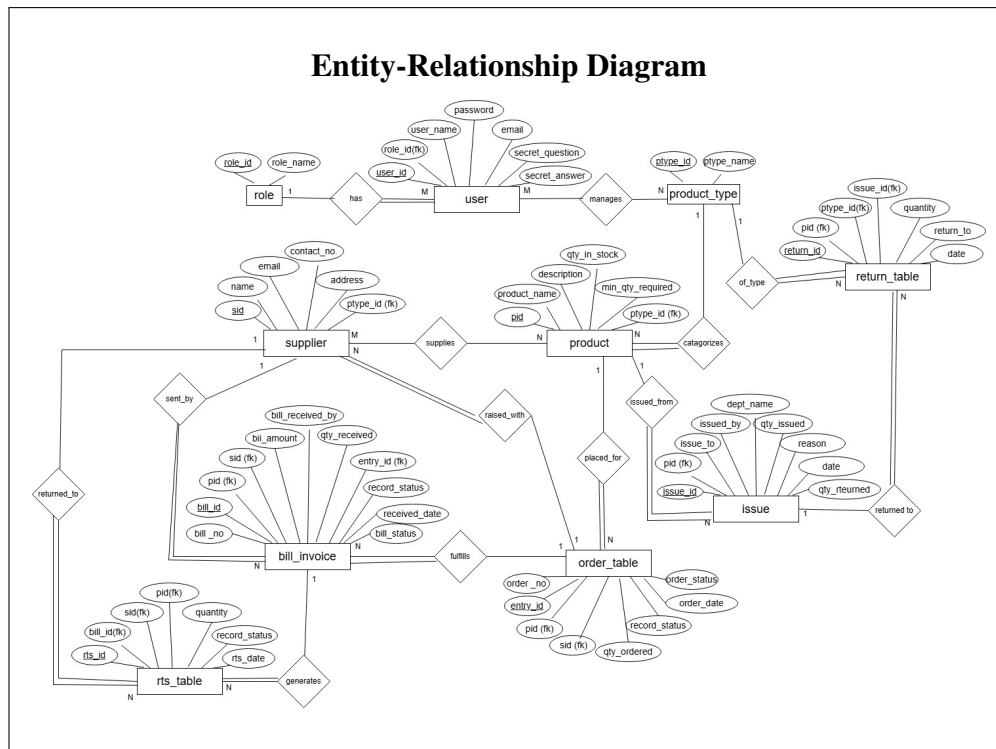


Figure 3.1: Entity-Relationship Diagram for IMA Database

3.4 Database Design

The database is implemented in MySQL 8.0 and normalised to the Third Normal Form (3NF). All tables use appropriate primary keys, foreign key constraints where applicable, and `record_status` flags for soft-delete support.

Table 3.2: User Table Schema

Field	Data Type	Key	Description
user_id	VARCHAR(10)	PK	Unique user identifier (e.g. M01, H01, D01)
password	VARCHAR(255)		User password (plain-text; hashing recommended for production)
role_id	INT	FK	Foreign key to Role table
secret_answer	VARCHAR(255)		Answer for password recovery

Table 3.3: Role Table Schema

Field	Data Type	Key	Description
role_id	INT	PK, AUTO	Unique role identifier
role_name	VARCHAR(20)	NOT NULL	Manager / HOD / Dean

Table 3.4: Product Table Schema

Field	Data Type	Key	Description
pid	VARCHAR(10)	PK	Unique product identifier
product_name	VARCHAR(100)	NOT NULL	Name of product
qty_in_stock	INT	NOT NULL	Current available quantity
min_qty	INT	NOT NULL	Minimum threshold for alert
ptype_id	INT	FK	Foreign key to Product_Type
record_status	VARCHAR(10)		ACTIVE / INACTIVE

Table 3.5: Product_Type Table Schema

Field	Data Type	Key	Description
ptype_id	INT	PK, AUTO	Unique type identifier
type_name	VARCHAR(50)	NOT NULL	Stationery / Furniture / Computer Set / Miscellaneous

Table 3.6: Supplier Table Schema

Field	Data Type	Key	Description
sid	INT	PK, AUTO	Unique supplier identifier
name	VARCHAR(100)	NOT NULL	Supplier company name
contact_no	VARCHAR(15)		Supplier contact number
email	VARCHAR(100)		Supplier email address
address	VARCHAR(255)		Supplier address

Table 3.7: Order Table Schema

Field	Data Type	Key	Description
entry_id	INT	PK, AUTO	Unique order entry
order_no	VARCHAR(50)	NOT NULL	Order number
pid	VARCHAR(10)	FK	Foreign key to Product
sid	INT	FK	Foreign key to Supplier
qty_ordered	INT	NOT NULL	Quantity ordered
order_date	DATE		Date of order placement
order_status	VARCHAR(15)		IN_PROCESS / PAID
record_status	VARCHAR(10)		ACTIVE / INACTIVE

Table 3.8: Bill_Invoice Table Schema

Field	Data Type	Key	Description
bill_id	INT	PK, AUTO	Unique bill identifier
bill_no	VARCHAR(50)	NOT NULL	Physical bill number
entry_id	INT	FK	Foreign key to Order
pid	VARCHAR(10)	FK	Foreign key to Product
sid	INT	FK	Foreign key to Supplier
qty_received	INT	NOT NULL	Quantity received in this bill
bill_amount	DECIMAL(10,2)	NOT NULL	Total bill amount
received_date	DATE	NOT NULL	Date of receipt
bill_received_by	VARCHAR(100)		Receiving person
bill_status	VARCHAR(15)		INCOMPLETE / COMPLETE
record_status	VARCHAR(10)		ACTIVE / INACTIVE

Table 3.9: Issue Table Schema

Field	Data Type	Key	Description
issue_id	INT	PK, AUTO	Unique issue record
pid	VARCHAR(10)	FK	Foreign key to Product
issue_to	VARCHAR(100)	NOT NULL	Faculty/person receiving item
issued_by	VARCHAR(10)	FK	Manager user_id
dept_name	VARCHAR(100)		Department name
qty_issued	INT	NOT NULL	Quantity issued
qty_returned	INT		Quantity returned so far (default 0)
reason	VARCHAR(255)		Reason for issuance
date	DATE	NOT NULL	Date of issuance

Table 3.10: Return_Table Schema

Field	Data Type	Key	Description
return_id	INT	PK, AUTO	Unique return record
ptype_id	INT	FK	Foreign key to Product_Type
issue_id	INT	FK	Foreign key to Issue record
pid	VARCHAR(10)	FK	Foreign key to Product
quantity	INT	NOT NULL	Quantity returned
return_to	VARCHAR(100)		Return destination (Inventory)
date	DATE	NOT NULL	Date of return

Table 3.11: RTS_Table Schema

Field	Data Type	Key	Description
rts_id	INT	PK, AUTO	Unique RTS record
bill_id	INT	FK	Foreign key to Bill_Invoice
pid	VARCHAR(10)	FK	Foreign key to Product
sid	INT	FK	Foreign key to Supplier
quantity	INT	NOT NULL	Quantity returned to supplier
rts_date	DATE	NOT NULL	Date of return to supplier
record_status	VARCHAR(10)		SENT / RECEIVED

3.5 Data Flow Diagrams

3.5.1 DFD Level 0 — Context Diagram

The context diagram shows the IMA as a single central process interacting with its external entities: Inventory Manager, HOD, Dean, and Faculty (as recipient of issued items). It provides a high-level view of the system's boundaries and key data flows without exposing internal sub-processes.

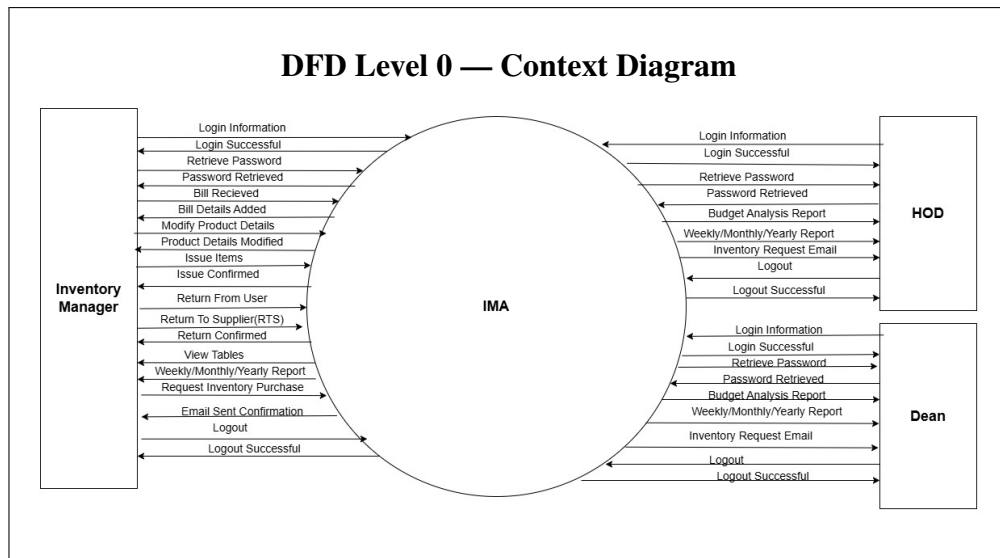


Figure 3.2: DFD Level 0 — Context Diagram for IMA

3.5.2 DFD Level 1 — Inventory Manager

The Level 1 DFD for the Inventory Manager decomposes the system into 15 sub-processes. Key processes include Register (1.1), Login (1.2), Retrieve Password (1.3), Add Order (1.4), Add Bill / Receive Inventory (1.5), View Details (1.6), Issue Inventory (1.7), Return Inventory (1.8), RTS (1.9), View Reports (1.10), Request Inventory Purchase (1.11), Low Stock Alert (1.12), View Notifications (1.13), View User Manual (1.14), and Logout (1.15).

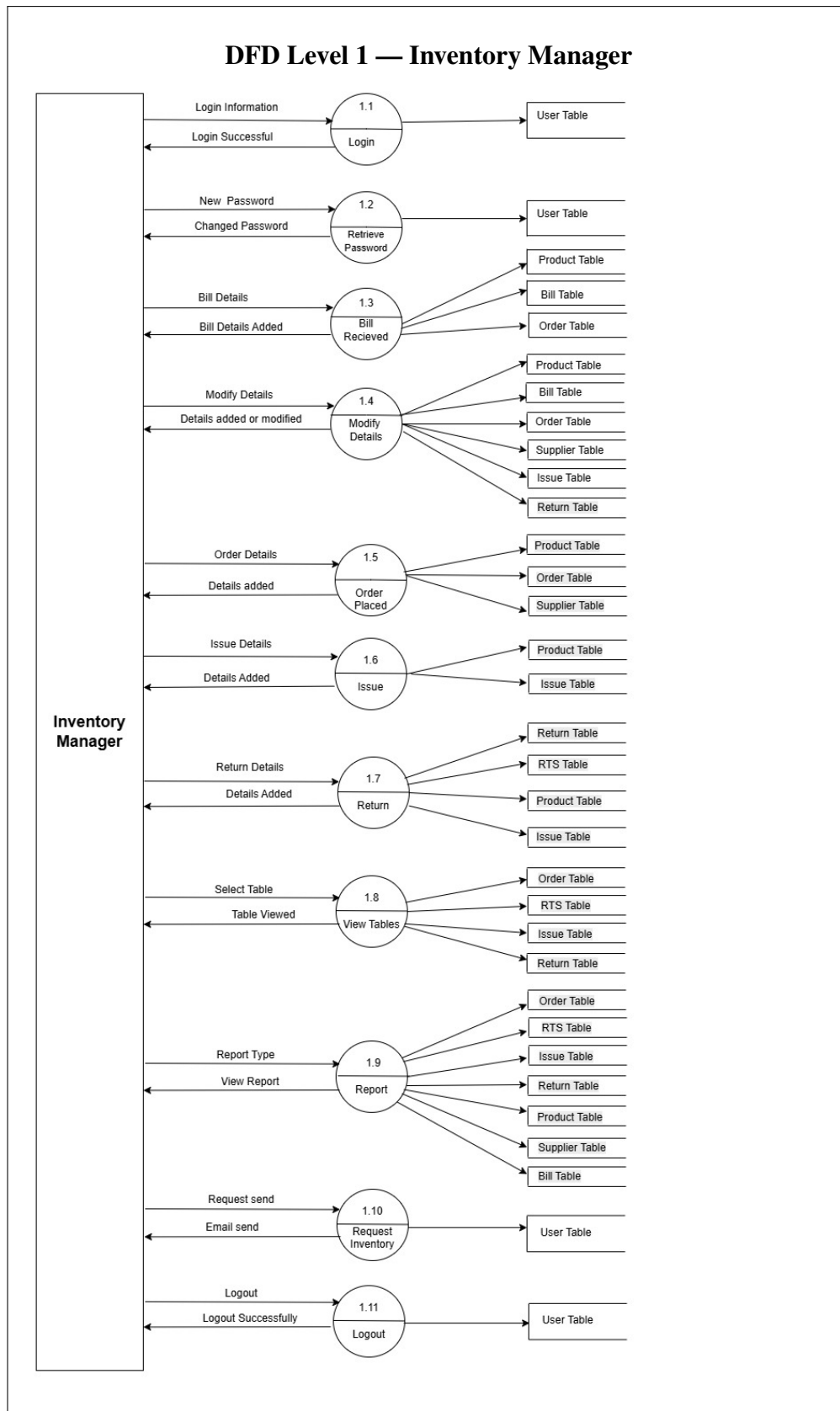


Figure 3.3: DFD Level 1 — Inventory Manager Processes

3.5.3 DFD Level 1 — HOD and Dean

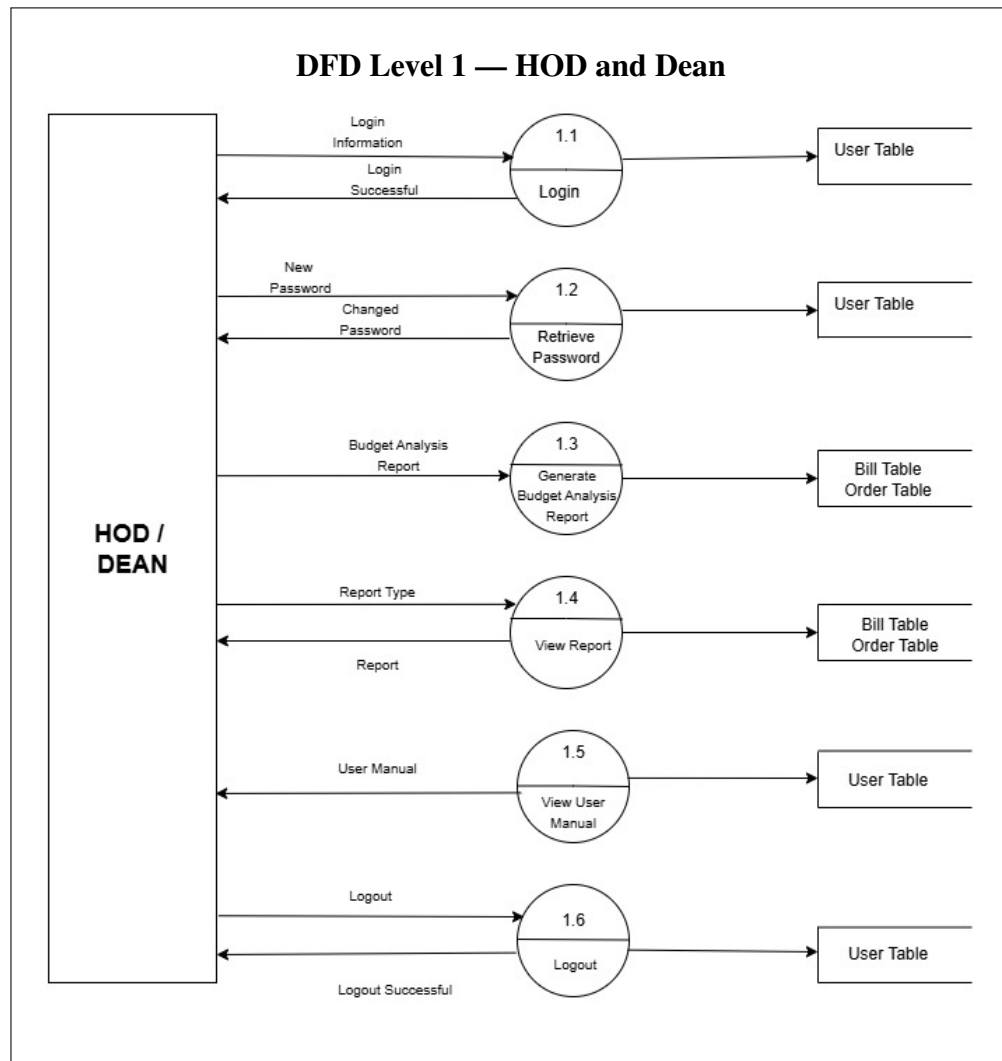
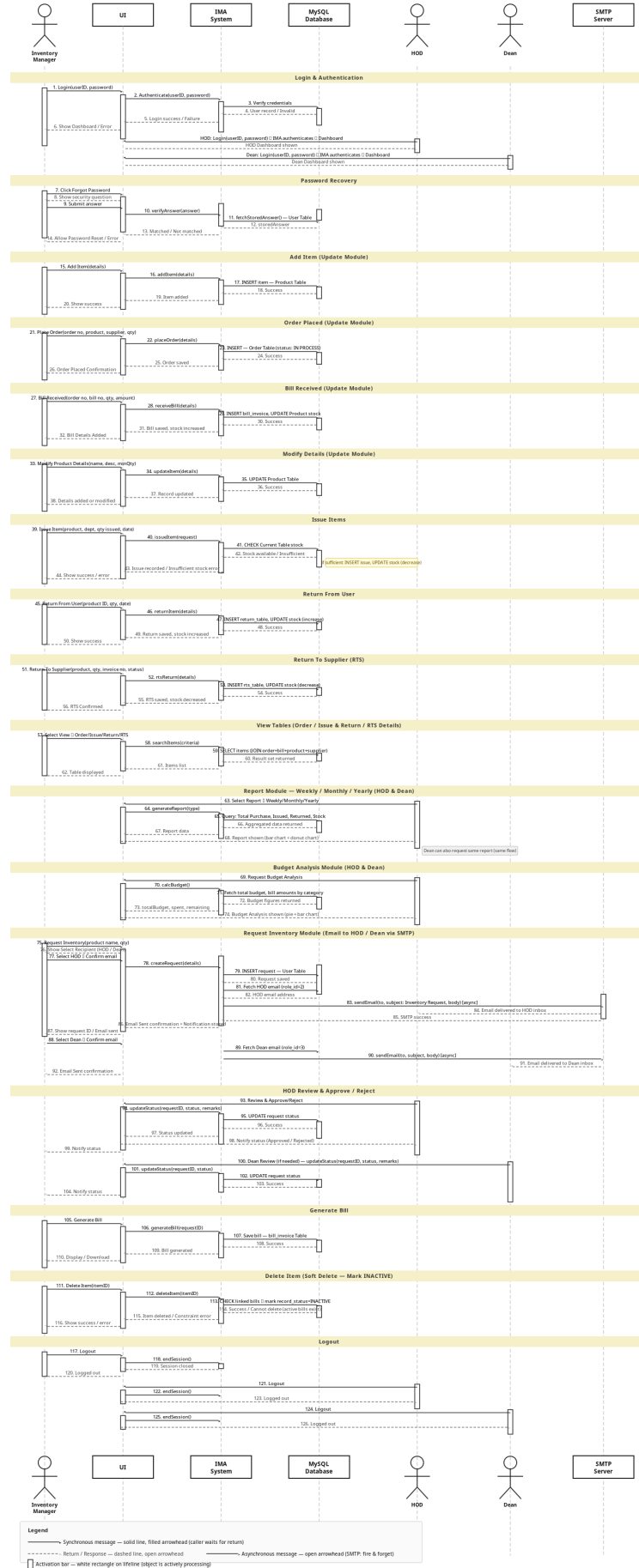


Figure 3.4: DFD Level 1 — HOD and Dean Processes

3.6 Sequence Diagrams

Sequence diagrams illustrate the time-ordered interactions between the system's objects and actors for the most critical use cases of the IMA. Each diagram shows the lifelines of participating components, the messages exchanged between them, and the sequence in which those messages occur. The diagrams presented in this section cover the Login process, the Add Bill / Receive Inventory process, the Issue Inventory process, and the Return (Faculty to Inventory) process.



30
Figure 3.5: Update Button — IMA

Chapter 4

Coding

This chapter presents the source code for the main modules of the Inventory Management Application, taken directly from the project GitHub . Only core functional modules are included here in accordance with university guidelines; the complete codebase is available at the repository link above. Delete and Modify operations are excluded from this chapter as they involve direct database record manipulation that is covered in the design documentation.

4.1 Module 1: Notification Service (NotificationService.java)

The `NotificationService` class implements an in-memory, application-scoped notification storage mechanism using JavaFX's `ObservableList`. It provides a static, shared list of notification message strings that any module in the application can write to by calling `addNotification()`. The `DashboardUI` reads this list when the user opens the notification panel from the profile icon in the top navigation bar.

```
1 package service;
2
3 import javafx.collections.FXCollections;
4 import javafx.collections.ObservableList;
5
6 public class NotificationService {
7
8     private static ObservableList<String> notifications =
9         FXCollections.observableArrayList();
10
11     public static void addNotification(String msg) {
12         notifications.add(msg);
13     }
14
15     public static ObservableList<String> getNotifications() {
16         return notifications;
17     }
18 }
```

Listing 4.1: NotificationService.java — In-App Notification Service (from GitHub)

The use of `FXCollections.observableArrayList()` means that any JavaFX UI component bound to this list would automatically reflect additions

in real time. The low-stock alert system and the purchase request workflow both call `addNotification()` to record events that the user can review later. The notification list is maintained in memory for the duration of the application session; it is not persisted to the database, which is a known limitation identified for future enhancement.

4.2 Module 2: Login and Authentication (LoginUI.java)

The `LoginUI` class implements the login screen and the complete three-step password recovery workflow. Authentication is performed by querying the `user` table joined with the `role` table using a parameterised prepared statement. Upon success, the user's role is stored globally in `Inventory.setUserRole()` and the user ID in `Inventory.setLoggedInUser()`, which are then used by the `DashboardUI` to apply role-based access control.

```
1 // Core authentication: joins user and role tables
2 private boolean authenticate(String userId, String pass) {
3     String sql = ""
4         SELECT r.role_name
5         FROM user u
6         JOIN role r ON u.role_id = r.role_id
7         WHERE u.user_id = ? AND u.password = ?
8         "";
9     try (Connection con = DBConnection.getConnection();
10         PreparedStatement ps = con.prepareStatement(sql)) {
11         ps.setString(1, userId);
12         ps.setString(2, pass);
13         ResultSet rs = ps.executeQuery();
14         if (rs.next()) {
15             String role = rs.getString("role_name");
16             Inventory.setUserRole(role);
17             Inventory.setLoggedInUser(userId);
18             return true;
19         }
20         return false;
21     } catch (Exception e) {
22         e.printStackTrace();
23         return false;
24     }
25 }
26
27 // Step 1: Verify that the User ID exists in the database
28 private boolean userExists(String userId) {
29     String sql = "SELECT 1 FROM user WHERE user_id = ?";
30     try (Connection con = DBConnection.getConnection());
```

```

31         PreparedStatement ps = con.prepareStatement(sql)) {
32             ps.setString(1, userId);
33             return ps.executeQuery().next();
34         } catch (Exception e) { return false; }
35     }
36
37     // Step 2: Verify the secret security answer
38     private boolean verifySecretAnswer(String userId, String ans) {
39         String sql =
40             "SELECT 1 FROM user WHERE user_id=? " +
41             "AND secret_answer=?";
42         try (Connection con = DBConnection.getConnection();
43             PreparedStatement ps = con.prepareStatement(sql)) {
44             ps.setString(1, userId);
45             ps.setString(2, ans);
46             return ps.executeQuery().next();
47         } catch (Exception e) { return false; }
48     }
49
50     // Step 3: Update password in database
51     private void showChangePasswordUI() {
52         Stage stage = new Stage();
53         PasswordField newPass = new PasswordField();
54         PasswordField confirm = new PasswordField();
55         Label status = new Label();
56         Button change = new Button("Update Password");
57         change.setOnAction(e -> {
58             if (!newPass.getText().equals(confirm.getText())) {
59                 status.setText("Passwords do not match.");
60                 return;
61             }
62             String sql = "UPDATE user SET password = ?" +
63                 " WHERE user_id = ?";
64             try (Connection con = DBConnection.getConnection();
65                 PreparedStatement ps =
66                     con.prepareStatement(sql)) {
67                 ps.setString(1, newPass.getText().trim());
68                 ps.setString(2,
69                     currentUserForPasswordChange);
70                 ps.executeUpdate();
71                 status.setText("Password updated successfully.");
72                 change.setDisable(true);
73             } catch (Exception ex) {
74                 status.setText("Error updating password.");
75             }
76         });
77     }

```

Listing 4.2: LoginUI.java — Authentication and Password Recovery (from GitHub)

The `authenticate()` method uses a JOIN query rather than two separate queries, ensuring that only users with a valid role assignment can log in. The use of `PreparedStatement` throughout all database interactions prevents SQL injection attacks. The three-step password recovery flow (User ID verification, secret answer verification, new password setting) ensures that password resets cannot be performed without knowing the registered security answer.

4.3 Module 3: Add Inventory / Bill Receipt (AddInventoryUI.java)

The `AddInventoryUI` class handles the receipt of inventory against an existing purchase order. This is one of the most complex modules in the application, as it must coordinate multiple database tables (`order_table`, `bill_invoice`, `product`) and enforce the business rule that a bill's quantity cannot exceed the remaining order balance. The `reloadOrderDropDown()` method ensures that fully paid orders are automatically removed from the dropdown after each successful bill entry.

```
1 // Fetch order details for the selected order number
2 PreparedStatement getOrder =
3     con.prepareStatement(
4         "SELECT entry_id, pid, sid, " +
5         "qty_ordered, order_status " +
6         "FROM order_table " +
7         "WHERE order_no = ? " +
8         "AND record_status = 'ACTIVE' LIMIT 1");
9 getOrder.setString(1, orderNo.getValue().trim());
10 ResultSet rs = getOrder.executeQuery();
11 if (!rs.next())
12     throw new Exception("Order not found!");
13
14 String status = rs.getString("order_status");
15 if ("PAID".equals(status))
16     throw new Exception("Order already PAID.");
17
18 int entryId    = rs.getInt("entry_id");
19 String pid     = rs.getString("pid");
20 int sid       = rs.getInt("sid");
21 int qtyOrdered = rs.getInt("qty_ordered");
22
23 // Calculate total already received for this order
24 PreparedStatement calcAlready =
25     con.prepareStatement(
26         "SELECT IFNULL(SUM(qty_received), 0) " +
```

```

27         " AS already FROM bill_invoice " +
28         "WHERE entry_id = ? " +
29         "AND record_status = 'ACTIVE'");
30 calcAlready.setInt(1, entryId);
31 ResultSet rsA = calcAlready.executeQuery();
32 rsA.next();
33 int alreadyReceived = rsA.getInt("already");
34 int remaining = qtyOrdered - alreadyReceived;
35
36 if (qty > remaining)
37     throw new Exception(
38         "Qty exceeds remaining limit (" + remaining + ")");
39
40 // Insert bill record and update stock
41 PreparedStatement insertBill = con.prepareStatement(
42     "INSERT INTO bill_invoice " +
43     "(bill_no, bill_received_by, bill_amount, " +
44     " pid, sid, entry_id, qty_received, " +
45     " received_date, bill_status, record_status) " +
46     "VALUES (?, 'Inventory Manager', ?, ?, ?, ?, ?, ?, " +
47     " 'INCOMPLETE', 'ACTIVE')");
48 insertBill.executeUpdate();
49
50 PreparedStatement updateStock = con.prepareStatement(
51     "UPDATE product SET " +
52     "qty_in_stock = qty_in_stock + ? WHERE pid = ?");
53 updateStock.setInt(1, qty);
54 updateStock.setString(2, pid);
55 updateStock.executeUpdate();
56
57 // Determine and set final statuses
58 int totalReceived = alreadyReceived + qty;
59 String newBillStatus =
60     (totalReceived >= qtyOrdered) ? "COMPLETE" : "INCOMPLETE";
61 String newOrderStatus =
62     (totalReceived >= qtyOrdered) ? "PAID" : "IN_PROCESS";

```

Listing 4.3: AddInventoryUI.java — Bill Receipt with Order Validation (from GitHub)

The `IFNULL(SUM(...), 0)` aggregation query correctly handles the edge case where no bills have been received yet for a new order, returning zero instead of `NULL`. The remaining balance calculation (`remaining = qtyOrdered - alreadyReceived`) ensures that partial deliveries are correctly tracked across multiple bill entries before the order is finally marked as `PAID`.

4.4 Module 4: Issue Inventory (IssueUI.java)

The IssueUI class manages the issuance of inventory items to faculty members. It enforces role-based access, validates all mandatory fields, performs a live stock check, and uses a JDBC atomic transaction to simultaneously insert the issuance record and decrement the product stock.

```
1 try (Connection con = DBConnection.getConnection()) {
2     con.setAutoCommit(false); // Begin atomic transaction
3
4     // Check current stock with a live database query
5     PreparedStatement check =
6         con.prepareStatement(
7             "SELECT qty_in_stock FROM product WHERE pid = ?");
8     check.setString(1, productBox.getValue().getPid());
9     ResultSet rs = check.executeQuery();
10    rs.next();
11    int stock = rs.getInt("qty_in_stock");
12
13    if (quantity > stock) {
14        new Alert(Alert.AlertType.ERROR,
15            "Insufficient stock! Available: " + stock).show();
16        con.rollback(); // Rollback on validation failure
17        return;
18    }
19
20    // Insert issue record into issue table
21    PreparedStatement insert =
22        con.prepareStatement(
23            "INSERT INTO issue (pid, issue_to, issued_by," +
24            " dept_name, qty_issued, reason, date)" +
25            " VALUES (?, ?, ?, ?, ?, ?, ?)");
26    insert.setString(1, productBox.getValue().getPid());
27    insert.setString(2, issueTo.getText());
28    insert.setString(3, issuedBy.getValue());
29    insert.setString(4, dept.getText());
30    insert.setInt(5, quantity);
31    insert.setString(6, reason.getText());
32    insert.setDate(7, Date.valueOf(date.getValue()));
33    insert.executeUpdate();
34
35    // Atomically decrement product stock
36    PreparedStatement update =
37        con.prepareStatement(
38            "UPDATE product SET " +
39            "qty_in_stock = qty_in_stock - ? WHERE pid = ?");
40    update.setInt(1, quantity);
```

```

41     update.setString(2, productBox.getValue().getPid());
42     update.executeUpdate();
43
44     con.commit(); // Commit --- both operations succeed together
45     new Alert(Alert.AlertType.INFORMATION,
46         "Product Issued Successfully").show();
47 }

```

Listing 4.4: IssueUI.java — Atomic Issue Transaction (from GitHub)

The atomic transaction pattern (`setAutoCommit(false)` followed by `commit()`) ensures that the issue record insertion and stock decrement always succeed or fail together. If a database exception occurs between the two SQL operations, the `catch` block calls `rollback()` to undo any partial changes, maintaining database consistency. The role-based access check at the start of the constructor ensures that only users with the Manager role can access this screen.

4.5 Module 5: Return Inventory (ReturnUI.java)

The `ReturnUI` class supports two distinct return workflows: faculty-to-inventory returns and inventory-to-supplier returns (RTS). The faculty-to-inventory return uses a JavaFX `TableView` with embedded editable fields — the return quantity is entered directly in the table row, and each row has a `CheckBox` for selection. The RTS workflow uses a form-based interface with product and bill dropdowns.

```

1  // Faculty to Inventory return --- atomic transaction
2  save.setOnAction(e -> {
3      try (Connection con = DBConnection.getConnection()) {
4          con.setAutoCommit(false);
5          boolean anySelected = false;
6
7          for (IssueRow row : list) {
8              if (!row.isSelected()) continue;
9              anySelected = true;
10
11             int qtyToReturn = row.getQtyReturn();
12             int maxReturnable =
13                 row.getQtyIssued() - row.getAlreadyReturned();
14
15             if (qtyToReturn <= 0 || qtyToReturn > maxReturnable)
16             {
17                 con.rollback();
18                 new Alert(Alert.AlertType.ERROR,
19                     "Invalid return quantity for: "
20                     + row.getPid()).show();

```

```

20         return;
21     }
22
23     // 1. Insert into return_table
24     PreparedStatement psInsert = con.prepareStatement (
25         "INSERT INTO return_table " +
26         " (ptype_id, issue_id, pid, quantity," +
27         "  return_to, date) " +
28         "VALUES " +
29         " ((SELECT ptype_id FROM product WHERE pid=?),"
+
30         "  ?, ?, ?, 'Inventory', ?)");
31     psInsert.setString(1, row.getPid());
32     psInsert.setInt(2, row.getIssueId());
33     psInsert.setString(3, row.getPid());
34     psInsert.setInt(4, qtyToReturn);
35     psInsert.setDate(5,
36         Date.valueOf(sharedReturnDate.getValue()));
37     psInsert.executeUpdate();
38
39     // 2. Increment product stock
40     PreparedStatement psStock = con.prepareStatement (
41         "UPDATE product " +
42         "SET qty_in_stock = qty_in_stock + ? " +
43         "WHERE pid = ?");
44     psStock.setInt(1, qtyToReturn);
45     psStock.setString(2, row.getPid());
46     psStock.executeUpdate();
47
48     // 3. Update qty_returned on the original issue row
49     PreparedStatement psIssue = con.prepareStatement (
50         "UPDATE issue " +
51         "SET qty_returned = qty_returned + ? " +
52         "WHERE issue_id = ?");
53     psIssue.setInt(1, qtyToReturn);
54     psIssue.setInt(2, row.getIssueId());
55     psIssue.executeUpdate();
56     }
57
58     con.commit();
59     new Alert(Alert.AlertType.INFORMATION,
60         "Return saved successfully!").show();
61 }
62 });

```

Listing 4.5: ReturnUI.java — Atomic Return Transaction (from GitHub)

The return transaction involves three coordinated database operations for each selected row: inserting a return record, incrementing stock, and updating the issue record's returned quantity. All three are committed atomically for all selected rows in a single transaction. The table is refreshed after a successful save to show only rows with remaining returnable quantities, preventing duplicate return entries.

4.6 Module 9: Report Generation (ReportUI.java)

The `ReportUI` class generates Weekly, Monthly, and Yearly inventory reports with bar charts and pie charts. It accepts a period string ("Weekly", "Monthly", or "Yearly"), computes the appropriate date range via `getDateRange()`, and queries the database for purchase, issued, return, and RTS quantities per inventory category. The report can be saved as a JPG image using a JavaFX snapshot.

```
1 // Compute start and end dates based on selected period
2 private LocalDate[] getDateRange(String period) {
3     LocalDate today = LocalDate.now();
4     switch (period) {
5         case "Weekly": {
6             LocalDate start = today.with(
7                 java.time.temporal.TemporalAdjusters
8                     .previous(java.time.DayOfWeek.SUNDAY));
9             return new LocalDate[]{start, start.plusDays(6)};
10        }
11        case "Monthly": {
12            LocalDate start = today.withDayOfMonth(1);
13            LocalDate end = today.with(
14                java.time.temporal.TemporalAdjusters
15                    .lastDayOfMonth());
16            return new LocalDate[]{start, end};
17        }
18        case "Yearly": {
19            LocalDate start =
20                LocalDate.of(today.getYear(), 1, 1);
21            LocalDate end =
22                LocalDate.of(today.getYear(), 12, 31);
23            return new LocalDate[]{start, end};
24        }
25        default:
26            return new LocalDate[]{
27                LocalDate.of(2020, 1, 1), today};
28    }
29 }
```

Listing 4.6: ReportUI.java — Date Range Computation and Report Data Loading (from GitHub)


```

1 // Load report data for each inventory category
2 private Map<String, model.ReportData> loadReport(String period)
3 {
4     Map<String, model.ReportData> map = new HashMap<>();
5     LocalDate[] range = getDateRange(period);
6
7     try (Connection con = db.DBConnection.getConnection()) {
8         for (String cat : categories) {
9             model.ReportData rd = new model.ReportData();
10
11             // Total items purchased (received via bills)
12             rd.purchase = getSum(con,
13                 "SELECT COALESCE(SUM(b.qty_received), 0) " +
14                 "FROM bill_invoice b " +
15                 "JOIN product p ON b.pid = p.pid " +
16                 "JOIN product_type pt " +
17                 "  ON p.ptype_id = pt.ptype_id " +
18                 "WHERE LOWER(pt.ptype_name) LIKE LOWER(?) " +
19                 "AND b.record_status = 'ACTIVE' " +
20                 "AND b.received_date BETWEEN ? AND ?",
21                 "%" + cat + "%", range);
22
23             // Total items issued to faculty
24             rd.issued = getSum(con,
25                 "SELECT COALESCE(SUM(i.qty_issued), 0) " +
26                 "FROM issue i " +
27                 "JOIN product p ON i.pid = p.pid " +
28                 "JOIN product_type pt " +
29                 "  ON p.ptype_id = pt.ptype_id " +
30                 "WHERE LOWER(pt.ptype_name) LIKE LOWER(?) " +
31                 "AND i.date BETWEEN ? AND ?",
32                 "%" + cat + "%", range);
33
34             // Items returned to inventory
35             // (UNION: faculty returns + supplier returns
36             // received)
37             rd.returnToInventory = getSum(con,
38                 "SELECT COALESCE(SUM(x.quantity), 0) " +
39                 "FROM ( " +
40                 "  SELECT r.quantity, p.ptype_id, " +
41                 "         r.date AS dt " +
42                 "  FROM return_table r " +
43                 "  JOIN product p ON r.pid = p.pid " +
44                 "  UNION ALL " +
45                 "  SELECT rts.quantity, p.ptype_id, " +
46                 "         rts.rts_date AS dt " +

```

```

45         " FROM rts_table rts " +
46         " JOIN product p ON rts.pid = p.pid " +
47         " WHERE rts.record_status = 'RECEIVED' " +
48         ") x " +
49         "JOIN product_type pt " +
50         " ON x.ptype_id = pt.ptype_id " +
51         "WHERE LOWER(pt.ptype_name) LIKE LOWER(?) " +
52         "AND x.dt BETWEEN ? AND ?",
53         "%" + cat + "%", range);
54
55         map.put(cat, rd);
56     }
57 } catch (Exception e) {
58     e.printStackTrace();
59 }
60 return map;
61 }

```

Listing 4.7: ReportUI.java — Category-wise Report Data Queries (from GitHub)

The `getSum()` helper method accepts a parameterised SQL string, a category pattern, and a date range array, executes the query, and returns the integer result. The UNION ALL subquery in the `returnToInventory` calculation correctly combines both faculty-to-inventory returns from `return_table` and supplier-to-inventory returns from `rts_table` where the status is RECEIVED. The report is rendered using a `BarChart` for aggregate movement totals and a `PieChart` (donut-style) for stock distribution by category. The `handleSave()` method uses `JavaFX snapshot()` to capture the full report layout and saves it as a JPEG file via `ImageIO.write()`.

4.7 Module 10: Budget Analysis (BudgetAnalysisUI.java)

The `BudgetAnalysisUI` class provides the HOD and Dean with a visual budget comparison between the current month and the previous month. It uses `BudgetService` to retrieve `BudgetData`, then renders three summary cards (Total Budget, Amount Spent, Remaining Balance), a donut `PieChart` showing current-month category distribution, and a grouped `BarChart` comparing current versus previous month per category.

```

1 // Create a comparison card: gradient header (current month)
2 // + white footer (previous month + delta percentage)
3 private VBox createComparisonCard(String title,
4                                   double curr,
5                                   double prev,
6                                   String gradient) {

```

```

7      // Gradient header section
8      Label titleLbl = new Label(title);
9      titleLbl.setStyle("-fx-text-fill: white; -fx-font-size: 16;"
10 );
11
12      Label monthLbl = new Label(CURR_LABEL);
13      monthLbl.setStyle(
14          "-fx-text-fill: rgba(255,255,255,0.80); -fx-font-size
15 :12;");
16
17      Label amountLbl = new Label("Rs." + fmt(curr));
18      amountLbl.setStyle(
19          "-fx-text-fill: white; -fx-font-size: 26;" +
20          " -fx-font-weight: bold;");
21
22      VBox header = new VBox(4, titleLbl, monthLbl, amountLbl);
23      header.setPadding(new Insets(20));
24      header.setStyle(
25          "-fx-background-radius: 20 20 0 0;" +
26          "-fx-background-color: linear-gradient(to right,"
27          + gradient + ");");
28
29      // White footer: previous month value + delta arrow
30      Label prevLbl =
31          new Label(PREV_LABEL + ": Rs." + fmt(prev));
32      prevLbl.setStyle("-fx-font-size: 13; -fx-text-fill: #666;");
33
34      boolean up = curr >= prev;
35      Label deltaLbl = new Label(
36          (up ? "Up " : "Down ") + pctChange(curr, prev));
37      deltaLbl.setStyle(
38          "-fx-font-size: 13; -fx-font-weight: bold;" +
39          "-fx-text-fill: " + (up ? "#2E7D32" : "#C62828") + ";");
40
41      HBox footer = new HBox(10, prevLbl, deltaLbl);
42      footer.setAlignment(Pos.CENTER_LEFT);
43      footer.setPadding(new Insets(14, 20, 14, 20));
44      footer.setStyle(
45          "-fx-background-color: white;" +
46          "-fx-background-radius: 0 0 20 20;");
47
48      VBox card = new VBox(header, footer);
49      card.setPrefWidth(300);
50      card.setStyle(
51          "-fx-background-radius: 20;" +
52          "-fx-effect: dropshadow(gaussian," +
53          " rgba(0,0,0,0.15), 25, 0, 0, 8);");

```

```

52     return card;
53 }

```

Listing 4.8: BudgetAnalysisUI.java — Summary Card with Month-over-Month Comparison (from GitHub)

```

1  // Build grouped bar chart: previous month (pastel)
2  // vs current month (solid), per category
3  private VBox createGroupedBarCard(BudgetData data) {
4      CategoryAxis xAxis = new CategoryAxis();
5      NumberAxis yAxis = new NumberAxis();
6      BarChart<String, Number> barChart =
7          new BarChart<>(xAxis, yAxis);
8      barChart.setLegendVisible(false);
9      barChart.setCategoryGap(25);
10     barChart.setBarGap(4);
11     barChart.setPrefHeight(320);
12
13     XYChart.Series<String, Number> currSeries =
14         new XYChart.Series<>();
15     XYChart.Series<String, Number> prevSeries =
16         new XYChart.Series<>();
17
18     Set<String> allCats = new LinkedHashSet<>();
19     allCats.addAll(data.getCategoryDistribution().keySet());
20     allCats.addAll(data.getPrevCategoryDistribution().keySet());
21
22     for (String cat : allCats) {
23         currSeries.getData().add(new XYChart.Data<>(cat,
24             data.getCategoryDistribution()
25                 .getOrDefault(cat, 0.0)));
26         prevSeries.getData().add(new XYChart.Data<>(cat,
27             data.getPrevCategoryDistribution()
28                 .getOrDefault(cat, 0.0)));
29     }
30
31     barChart.getData().addAll(currSeries, prevSeries);
32
33     // Apply solid colours to current, pastel to previous
34     String[] solidColors = { ORANGE, BLUE, PURPLE, GREEN };
35     String[] pastelColors = { ORANGE_P, BLUE_P, PURPLE_P,
36         GREEN_P };
37
38     Platform.runLater(() -> {
39         int i = 0;
40         for (XYChart.Data<String, Number> d : currSeries.getData
41             ()) {

```

```

40         if (d.getNode() != null)
41             d.getNode().setStyle("-fx-bar-fill:"
42                 + solidColors[i % solidColors.length] + ";")
43     ;
44     i++;
45 }
46 int j = 0;
47 for (XYChart.Data<String, Number> d : prevSeries.getData
48     ()) {
49     if (d.getNode() != null)
50         d.getNode().setStyle("-fx-bar-fill:"
51             + pastelColors[j % pastelColors.length] + ";
52 ");
53     j++;
54 }
55 });
56
57 Label heading = new Label(
58     "Category-wise: " + PREV_LABEL + " vs " + CURR_LABEL);
59 heading.setFont(
60     Font.font("Segoe UI", FontWeight.BOLD, 14));
61
62 VBox layout = new VBox(12, heading, barChart);
63 layout.setAlignment(Pos.CENTER);
64 return wrapInCard(layout);
65 }

```

Listing 4.9: BudgetAnalysisUI.java — Grouped Bar Chart for Month Comparison (from GitHub)

The BudgetAnalysisUI uses two `XYChart.Series` objects for the grouped bar chart — one for the current month (solid colours: orange, blue, purple, green) and one for the previous month (pastel variants of the same colours). `Platform.runLater()` is used to apply colour styles after the JavaFX chart nodes have been fully rendered. The `pctChange()` helper computes the month-over-month percentage change and formats it with a directional arrow indicator, which is displayed in green for positive change and red for negative change.

4.8 Module 7: Add Order (AddOrderUI.java)

The AddOrderUI class allows the Inventory Manager to place a new purchase order by selecting a product and supplier from database-populated drop-downs, entering a quantity, and specifying the order date. The order is saved in `order_table` with status `IN_PROCESS`. The class also prevents reuse of an order number that is already marked as `PAID`.

```

1 save.setOnAction(e -> {
2     try (Connection con = DriverManager.getConnection(
3         "jdbc:mysql://localhost:3306/inventory_system",
4         "root", "")) {
5
6         if (orderNo.getText().trim().isEmpty())
7             throw new Exception("Enter Order No");
8         if (productName.getValue() == null)
9             throw new Exception("Select a Product");
10        if (supplierName.getValue() == null)
11            throw new Exception("Select a Supplier");
12        if (qtyOrdered.getText().trim().isEmpty())
13            throw new Exception("Enter Quantity");
14        if (orderDate.getValue() == null)
15            throw new Exception("Select Order Date");
16
17        // Resolve product ID from selected name
18        PreparedStatement ps1 = con.prepareStatement(
19            "SELECT pid FROM product WHERE product_name=?");
20        ps1.setString(1, productName.getValue());
21        ResultSet rs1 = ps1.executeQuery();
22        if (!rs1.next())
23            throw new Exception("Product not found");
24        String pid = rs1.getString("pid");
25
26        // Resolve supplier ID from selected name
27        PreparedStatement ps2 = con.prepareStatement(
28            "SELECT sid FROM supplier WHERE name=?");
29        ps2.setString(1, supplierName.getValue());
30        ResultSet rs2 = ps2.executeQuery();
31        if (!rs2.next())
32            throw new Exception("Supplier not found");
33        int sid = rs2.getInt("sid");
34
35        // Prevent reuse of a PAID order number
36        PreparedStatement checkPaid = con.prepareStatement(
37            "SELECT order_status FROM order_table " +
38            "WHERE order_no=? AND record_status='ACTIVE' LIMIT 1
39        ");
40        checkPaid.setString(1, orderNo.getText().trim());
41        ResultSet rsPaid = checkPaid.executeQuery();
42        if (rsPaid.next() &&
43            "PAID".equals(rsPaid.getString("order_status")))
44            throw new Exception("Order No already PAID.");
45        // Insert new order record

```

```

46     PreparedStatement insertOrder = con.prepareStatement (
47         "INSERT INTO order_table " +
48         "(order_no, pid, sid, order_date, qty_ordered, " +
49         " order_status, record_status) " +
50         "VALUES (?, ?, ?, ?, ?, 'IN_PROCESS', 'ACTIVE')");
51     insertOrder.setString(1, orderNo.getText().trim());
52     insertOrder.setString(2, pid);
53     insertOrder.setInt(3, sid);
54     insertOrder.setDate(4,
55         java.sql.Date.valueOf(orderDate.getValue()));
56     insertOrder.setInt(5,
57         Integer.parseInt(qtyOrdered.getText().trim()));
58     insertOrder.executeUpdate();
59
60     new Alert(Alert.AlertType.INFORMATION,
61         "Order added successfully").show();
62
63     // Reset all form fields after successful save
64     orderNo.clear();
65     productName.setValue(null);
66     supplierName.setValue(null);
67     qtyOrdered.clear();
68     orderDate.setValue(null);
69
70 } catch (Exception ex) {
71     new Alert(Alert.AlertType.ERROR,
72         ex.getMessage()).show();
73 }
74 });

```

Listing 4.10: AddOrderUI.java — Place Order Logic (from GitHub)

The `setupComboBox()` helper populates both the product and supplier dropdowns by executing `SELECT` queries and loading results into the `ComboBox` items list. The order status is set to `IN_PROCESS` automatically on every new order and is only updated to `PAID` by the bill receipt module when the total received quantity equals the ordered quantity. Product and supplier IDs are resolved from their display names via subqueries, ensuring that the foreign keys stored in `order_table` are always valid.

4.9 Module 8: Manage Inventory (ManageInventoryUI.java — Key Sections)

The `ManageInventoryUI` class provides a tabbed interface for viewing and managing Products, Suppliers, Orders, and Bills. The Product and Supplier tabs support inline editing via `TextFieldTableCell`; changes are committed to the

database immediately on cell edit completion. The Order and Bill tabs are read-only with soft-delete support. Bill deletion triggers a three-step process: soft-delete, stock reversal, and order/bill status recalculation.

```
1 // Product name column: inline editable via TextFieldTableCell
2 nameCol.setCellFactory(TextFieldTableCell.forTableColumn());
3 nameCol.setOnEditCommit(e -> {
4     e.getRowValue().name.set(e.getNewValue());
5     updateProduct(e.getRowValue());
6 });
7
8 // Description column: inline editable
9 descCol.setCellFactory(TextFieldTableCell.forTableColumn());
10 descCol.setOnEditCommit(e -> {
11     e.getRowValue().desc.set(e.getNewValue());
12     updateProduct(e.getRowValue());
13 });
14
15 // Min Qty column: inline editable with integer converter
16 minCol.setCellFactory(TextFieldTableCell.forTableColumn(
17     new javafx.util.converter.IntegerStringConverter()));
18 minCol.setOnEditCommit(e -> {
19     e.getRowValue().minQty.set(e.getNewValue());
20     updateProduct(e.getRowValue());
21 });
22
23 // Update persisted to database immediately on edit commit
24 private void updateProduct(PRow row) {
25     String sql =
26         "UPDATE product SET product_name=?, qty_in_stock=?, " +
27         " description=?, min_qty_required=? WHERE pid=?";
28     try (Connection con = DBConnection.getConnection();
29         PreparedStatement ps = con.prepareStatement(sql)) {
30         ps.setString(1, row.name.get());
31         ps.setInt(2, row.qty.get());
32         ps.setString(3, row.desc.get());
33         ps.setInt(4, row.minQty.get());
34         ps.setString(5, row.pid.get());
35         ps.executeUpdate();
36     } catch (Exception e) { showError(e.getMessage()); }
37 }
```

Listing 4.11: ManageInventoryUI.java — Inline Edit on Product Table (from GitHub)

```
1 // Bill delete: soft-delete -> reverse stock -> recalculate
  statuses
2 btn.setOnAction(e -> {
3     BRow row = getTableView().getItems().get(getIndex());
```



```

4     if (confirmDelete("bill '" + row.billNo.get() + "'")) {
5
6         // Step 1: Mark bill as INACTIVE (soft-delete)
7         softDeleteBill(row.billId.get());
8
9         // Step 2: Reverse the stock that was added by this bill
10        reverseStock(row);
11
12        // Step 3: Recalculate order and bill statuses
13        //           based on remaining ACTIVE bills only
14        recalculateStatuses(row.entryId.get());
15
16        // Step 4: Reload both tables live
17        loadBills(table);
18        loadOrders(orderTable);
19    }
20 });
21
22 // Recalculate order/bill statuses after a bill is deleted
23 private void recalculateStatuses(int entryId) {
24     try (Connection con = DBConnection.getConnection()) {
25         PreparedStatement psOrd = con.prepareStatement(
26             "SELECT qty_ordered FROM order_table WHERE entry_id
27             =?");
28         psOrd.setInt(1, entryId);
29         ResultSet rsOrd = psOrd.executeQuery();
30         if (!rsOrd.next()) return;
31         int qtyOrdered = rsOrd.getInt("qty_ordered");
32
33         // Sum only ACTIVE (non-deleted) bills
34         PreparedStatement psSum = con.prepareStatement(
35             "SELECT IFNULL(SUM(qty_received), 0) AS total " +
36             "FROM bill_invoice " +
37             "WHERE entry_id=? AND record_status='ACTIVE'");
38         psSum.setInt(1, entryId);
39         ResultSet rsSum = psSum.executeQuery();
40         rsSum.next();
41         int totalReceived = rsSum.getInt("total");
42
43         // Derive new statuses in Java logic
44         String newOrderStatus =
45             (totalReceived >= qtyOrdered) ? "PAID" : "IN_PROCESS
46         ";
47         String newBillStatus =
48             (totalReceived >= qtyOrdered) ? "COMPLETE" : "
49         INCOMPLETE";

```

```

48         PreparedStatement psO = con.prepareStatement (
49             "UPDATE order_table SET order_status=? WHERE
entry_id=?");
50         psO.setString(1, newOrderStatus);
51         psO.setInt(2, entryId);
52         psO.executeUpdate();
53
54         PreparedStatement psB = con.prepareStatement (
55             "UPDATE bill_invoice SET bill_status=? " +
56             "WHERE entry_id=? AND record_status='ACTIVE'");
57         psB.setString(1, newBillStatus);
58         psB.setInt(2, entryId);
59         psB.executeUpdate();
60
61     } catch (Exception e) { showError(e.getMessage()); }
62 }

```

Listing 4.12: ManageInventoryUI.java — Bill Delete with Stock Reversal and Status Recalculation (from GitHub)

The bill deletion logic follows a strict three-step sequence to maintain data consistency. The `softDeleteBill()` method marks the bill record as `INACTIVE` rather than physically removing it, preserving the audit trail. The `reverseStock()` method decrements `product.qty_in_stock` by the bill's received quantity, restoring the stock to its pre-receipt level. The `recalculateStatuses()` method then recomputes the order status from scratch using only the remaining `ACTIVE` bills, correctly reverting the order from `PAID` back to `IN_PROCESS` if the deletion brings the total received below the ordered quantity.

4.10 Module 6: Dashboard (DashboardUI.java — Key Sections)

The `DashboardUI` class is the central hub of the application. It constructs the top navigation bar with greeting, date/time, profile icon (with contextual menu showing role, user ID, and notifications), User Manual link, and Logout button. The main content area features a `GridPane` of dashboard buttons whose visibility and enabled state are dynamically controlled based on the logged-in user's role.

```

1  // Role-based button visibility enforcement
2  String role = Inventory.getUserRole();
3
4  // HOD and Dean can only access Report and Budget Analysis
5  if (role.equalsIgnoreCase("HOD") ||
6      role.equalsIgnoreCase("Dean")) {
7      if (!btn.getText().equals("Report") &&
8          !btn.getText().equals("Budget Analysis")) {

```

```

9         btn.setDisable(true);
10        btn.setOpacity(0.5);
11    }
12 }
13
14 // Manager cannot access Budget Analysis
15 if (role.equalsIgnoreCase("Manager")) {
16     if (btn.getText().equals("Budget Analysis")) {
17         btn.setDisable(true);
18         btn.setOpacity(0.5);
19     }
20 }

```

Listing 4.13: DashboardUI.java — Role-Based Button Control (from GitHub)

```

1 // Live SQL query powering the Order Details table view
2 String query =
3     "SELECT p.product_name, " +
4     "ot.qty_ordered, " +
5     "ot.order_date, " +
6     "ot.order_status, " +
7     "bi.qty_received, " +
8     "bi.received_date, " +
9     "bi.bill_no, " +
10    "bi.bill_amount, " +
11    "bi.bill_status, " +
12    "s.name AS supplier_name, " +
13    "s.contact_no " +
14    "FROM order_table ot " +
15    "JOIN product p ON ot.pid = p.pid " +
16    "JOIN supplier s ON ot.sid = s.sid " +
17    "LEFT JOIN bill_invoice bi " +
18    "ON bi.entry_id = ot.entry_id " +
19    "AND bi.record_status = 'ACTIVE' " +
20    "WHERE ot.record_status = 'ACTIVE' " +
21    "ORDER BY ot.order_date DESC";

```

Listing 4.14: DashboardUI.java — Order Details Table Query (from GitHub)

The Order Details query uses a `LEFT JOIN` between `order_table` and `bill_invoice`, ensuring that orders for which no bill has been received yet still appear in the table with empty bill columns. Similarly, the Issue & Return Details query uses a `LEFT JOIN` with `return_table` to show all issues regardless of whether a return has been processed. The `ORDER BY order_date DESC` clause ensures the most recent entries appear first.

Chapter 5

Testing

5.1 Testing Strategy

Testing is a critical phase ensuring the application functions correctly and in accordance with specified requirements. The IMA was tested using a structured multi-phase strategy covering unit testing, integration testing, system testing, stress testing, and acceptance testing. All test cases were designed from the functional requirements defined in Chapter 2 and executed against a local MySQL test instance of the application with pre-populated test data.

The testing process was conducted iteratively alongside development. Each new module was unit-tested by its developer before integration. A complete integration and regression test suite was executed after each major feature addition to ensure that new functionality did not introduce regressions in existing modules.

5.2 Unit Testing

Unit tests verify individual methods, classes, and modules in isolation. Tests were conducted by each developer on local machines, focusing on data validation, database query correctness, role-based access enforcement, atomic transaction behaviour, and UI component behaviour.

Table 5.1: Unit Test Cases

TC#	Test Description	Input	Expected Output	Result
TC01	Valid login	Correct user_id & password	Dashboard shown; role set correctly	Pass
TC02	Invalid password	Correct user_id, wrong password	Error message displayed	Pass
TC03	Empty login fields	Both fields blank	Validation warning shown	Pass
TC04	Account lockout	3 wrong login attempts in sequence	Login disabled; lockout message shown	Pass
TC05	Password retrieval — valid User ID	Existing user_id entered	Proceeds to secret question screen	Pass

TC#	Test Description	Input	Expected Output	Result
TC06	Password retrieval — invalid User ID	Non-existent user_id	“User ID not found” error shown	Pass
TC07	Password retrieval — correct answer	Valid secret answer	Proceed to set new password screen	Pass
TC08	Password retrieval — wrong answer	Invalid secret answer	“Wrong answer. Try again.” shown	Pass
TC09	Password retrieval — mismatch	New and confirm passwords differ	“Passwords do not match.” shown	Pass
TC10	Add bill — valid data	Active order, valid qty $\leq$ remaining	Bill saved; stock incremented; statuses updated	Pass
TC11	Add bill — order PAID	PAID order selected	Error: order already PAID	Pass
TC12	Add bill — qty exceeds remaining	qty > order balance	Error: qty exceeds remaining limit shown	Pass
TC13	Add bill — empty bill no	Bill No field blank	Validation error shown	Pass
TC14	Add bill — partial receipt	qty received < qty ordered	bill_status = INCOMPLETE; order IN_PROCESS	Pass
TC15	Add bill — full receipt	qty received = remaining balance	bill_status = COMPLETE; order PAID	Pass
TC16	Order dropdown refresh	After bill saved successfully	PAID orders no longer appear in dropdown	Pass
TC17	Issue item — valid qty	qty $\leq$ available stock, all fields filled	Record inserted; stock decremented	Pass
TC18	Issue item — exceeds stock	qty > available stock	Error: Insufficient stock; no DB changes	Pass

TC#	Test Description	Input	Expected Output	Result
TC19	Issue item — zero qty	qty = 0 entered	Validation error: Enter valid quantity	Pass
TC20	Issue item — empty fields	Any mandatory field left blank	“All fields are mandatory” error	Pass
TC21	Issue role enforcement	Non-Manager user reaches IssueUI	Access Denied message; redirect to dashboard	Pass
TC22	Issue — atomic roll-back	DB failure simulated between insert and update	Both insert and update rolled back; stock unchanged	Pass
TC23	Return — valid selection	Row selected, valid return qty entered	Return recorded; stock incremented; issue updated	Pass
TC24	Return — qty exceeds returnable	qty > (issued – already returned)	Error: exceeds returnable qty; roll-back	Pass
TC25	Return — no row selected	Save clicked without selecting any row	Warning: select at least one row	Pass
TC26	RTS — valid	Product and bill selected, valid qty	RTS record saved; stock decremented	Pass
TC27	RTS — qty exceeds stock	qty > current stock for SENT status	Error: qty exceeds current stock	Pass
TC28	HOD login	HOD credentials	HOD dashboard with only Report and Budget Analysis enabled	Pass
TC29	Dean login	Dean credentials	Dean dashboard with only Report and Budget Analysis enabled	Pass

TC#	Test Description	Input	Expected Output	Result
TC30	Manager — Budget Analysis blocked	Manager logs in and views dashboard	Budget Analysis button disabled (opacity 0.5)	Pass
TC31	Logout	Logout label clicked	Session terminated; login screen displayed	Pass
TC32	Product load in Issue form	IssueUI opens	Product drop-down populated from product table	Pass
TC33	Notifications panel	Notification added; panel opened	Notification message displayed in panel	Pass
TC34	Order Details view	View → Order Details selected	All active orders with bill data shown in table	Pass
TC35	Issue & Return Details view	View → Issue & Return Details selected	All issues with return status displayed	Pass
TC36	RTS Details view	View → RTS Details selected	All RTS records displayed with supplier info	Pass
TC37	DBConnection URL	DBConnection.getConnection() called	Valid JDBC connection returned; no exception	Pass

5.3 Integration Testing

Integration tests confirmed that all modules interact correctly through the shared database and the global session state managed by the `Inventory` class.

The **Bill Receipt to Order Transition** integration test verified the end-to-end flow: the Manager adds a bill, the order status transitions from `IN_PROCESS` to `PAID` when fully received, and the dropdown is updated to remove the paid order. This test confirmed correct coordination between `AddInventoryUI`, the `order_table`, and `bill_invoice` tables.

The **Issue to Return** integration test verified that after issuing items, those items appear in the Return screen with the correct remaining returnable quantity. After processing a partial return, the issue record's `qty_returned` field was verified

to be updated correctly, and the stock level was confirmed to have increased by the returned amount.

The **Authentication to Dashboard Role Restriction** integration test verified that logging in as HOD correctly disables the Update, Issue, Return, and Request Inventory buttons, while logging in as Manager correctly disables the Budget Analysis button. This confirmed that role information stored in `Inventory.getUserRole()` is correctly read and applied by `DashboardUI`.

The **Notification Service** integration test verified that calling `NotificationService.addNotification()` from any module results in the message appearing in the notification panel accessible from the dashboard profile icon.

5.4 System Testing

System tests verified end-to-end workflows across all integrated modules executed in realistic usage sequences:

1. **Complete Procurement Cycle:** Manager logs in, places an order (`AddOrderUI`), receives a partial bill (`AddInventoryUI`), verifies order status remains `IN_PROCESS`, receives the remaining quantity in a second bill, verifies order transitions to `PAID` and disappears from the dropdown.
2. **Issue and Return Cycle:** Manager issues items to a faculty member, verifies stock decremented, processes a partial return, verifies stock incremented and issue record updated, processes the remaining return, verifies the issue row no longer appears in the pending returns table.
3. **RTS Cycle:** Manager selects a product and its associated bill, enters a return quantity, saves as `SENT` status, verifies stock decremented and `RTS` record appears in the `RTS Details` view.
4. **Password Recovery Cycle:** User enters valid User ID, answers secret question correctly, sets new password, logs out, logs in with new password successfully.
5. **Role Access Cycle:** All three roles log in sequentially. Manager sees all enabled buttons except Budget Analysis. HOD and Dean see only Report and Budget Analysis enabled. Budget Analysis chart renders with correct data.

5.5 Stress Testing

Three Inventory Manager accounts, one HOD account, and one Dean account were simultaneously logged in and performing operations including bill receipt, issuance, and report viewing. The dashboard loaded within 2.5 seconds for all concurrent users, within the 3-second performance requirement. No deadlocks were observed during concurrent operations, confirming the effectiveness of atomic transactions and per-operation JDBC connection management.

5.6 Acceptance Testing

Acceptance testing was conducted with the office staff of the Faculty of Mathematics and Computing as primary evaluators. All 37 unit test cases were re-executed in the presence of end users. The mentor, Dr. Mainaz Faridi, reviewed the complete system against the SRS. All defined functional requirements were verified as met, and the system was declared ready for deployment pending installation of JAVA 25 LTS and MySQL 8.0 on departmental machines.

Chapter 6

User Interfaces

The graphical user interface is built using JavaFX. The design uses a consistent warm brown-beige colour scheme (#E2C49F, #8B5E34) throughout all screens for a cohesive visual identity. All screens target a minimum resolution of 1024×768 pixels. Role-based restrictions are applied dynamically at runtime.

6.1 Login Screen

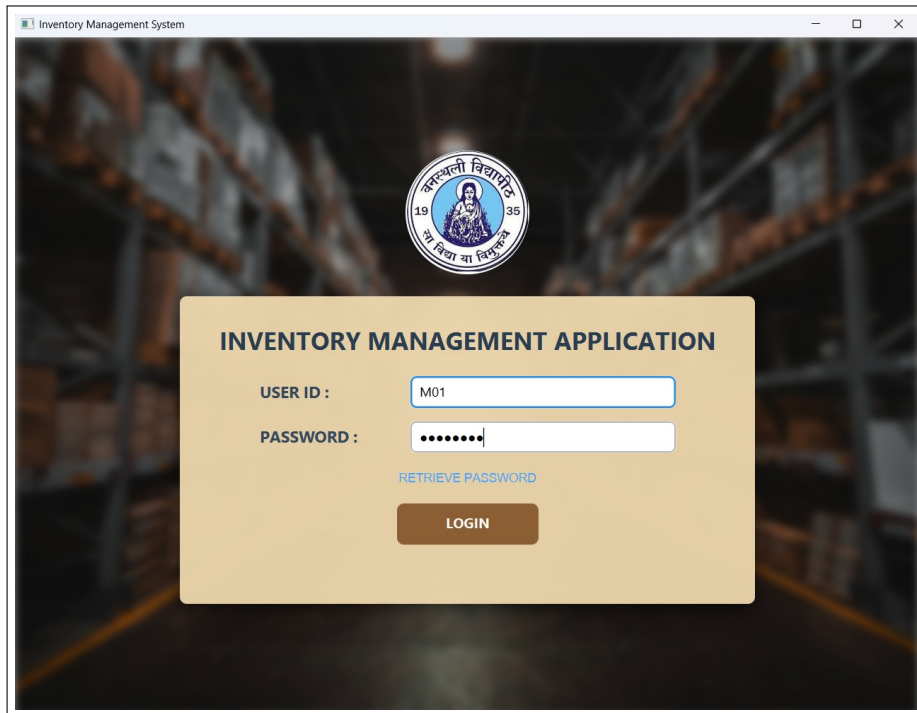


Figure 6.1: Login Screen — Inventory Management Application

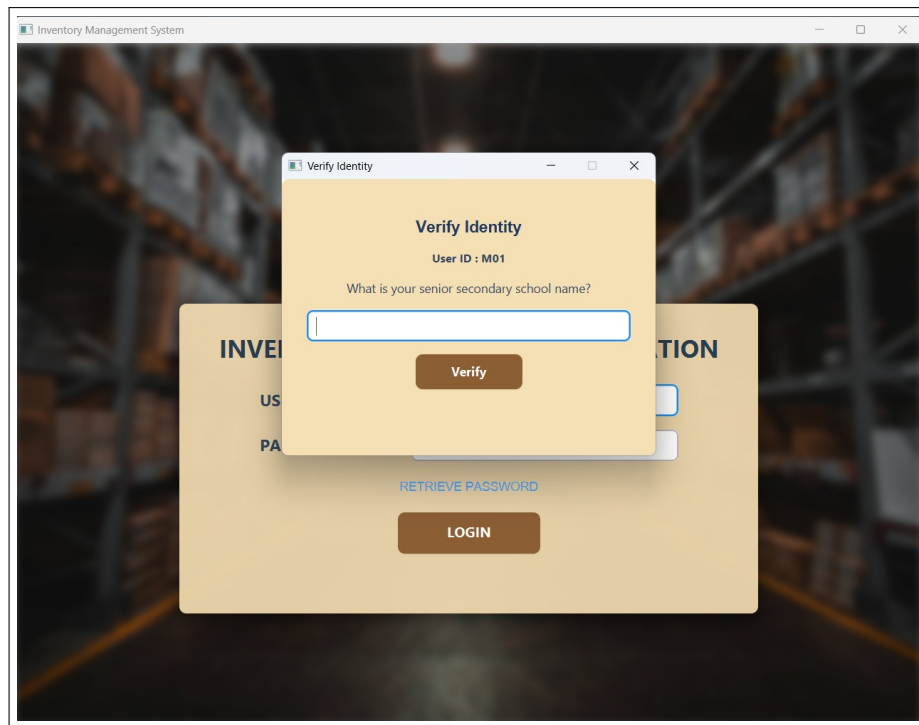


Figure 6.2: Retrieve Password Screen — Inventory Management Application

6.2 Main Dashboard

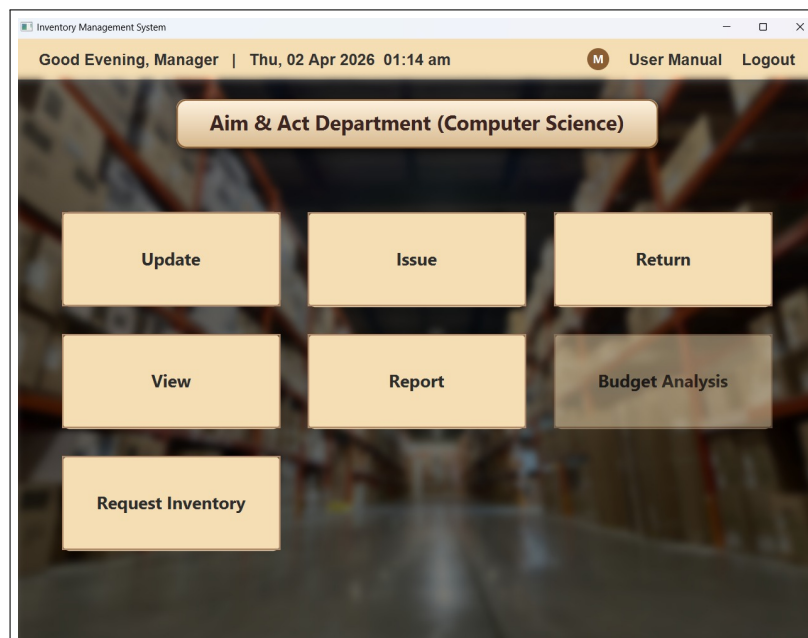


Figure 6.3: Main Dashboard — Inventory Management Application

6.3 Update

The screenshot shows the IMA interface. At the top, it says "Good Evening, Manager | Thu, 02". Below this is a navigation bar with "Aim & Act". A central menu lists "Update", "Bill Received", "Modify", "Order Placed", and "Request Inventory". The "Update" option is highlighted. Below the menu, the "BILL RECEIVED DETAILS" form is displayed. It contains fields for "Order No", "Bill No", "Bill Amount", "Received Date", and "Quantity Rece...". There are "SAVE" and "BACK" buttons at the bottom.

Inventory Management System

Good Evening, Manager | Thu, 02

Aim & Act

Update

Bill Received

Modify

Order Placed

Request Inventory

BILL RECEIVED DETAILS

Order No: [dropdown] Bill Amount: [text input]

Bill No: [text input] Received Date: [calendar icon]

Quantity Rece...: [text input]

SAVE BACK

Figure 6.4: Bill Recieved form — IMA

The screenshot shows the "ADD ORDER FORM" in the IMA. It contains fields for "Order No" (with value "OR0014"), "Order Date" (with calendar icon), "Product Name" (with dropdown), "Quantity Orde..." (with text input), "Supplier Name" (with dropdown), and "Order Status" (with value "IN PROCESS"). There are "SAVE" and "BACK" buttons at the bottom.

ADD ORDER FORM

Order No: OR0014 Order Date: [calendar icon]

Product Name: [dropdown] Quantity Orde...: [text input]

Supplier Name: [dropdown] Order Status: IN PROCESS

SAVE BACK

Figure 6.5: Add Order Form— IMA

The screenshot shows the "Manage Inventory Data" table in the IMA. It has tabs for "Product", "Supplier", "Orders", and "Bills". The "Product" tab is selected. The table has columns: "Name", "Description", "Min Qty", "Stock", and "Type". There is a search bar at the top right with the text "Search by name, desc".

Manage Inventory Data

Product Supplier Orders Bills

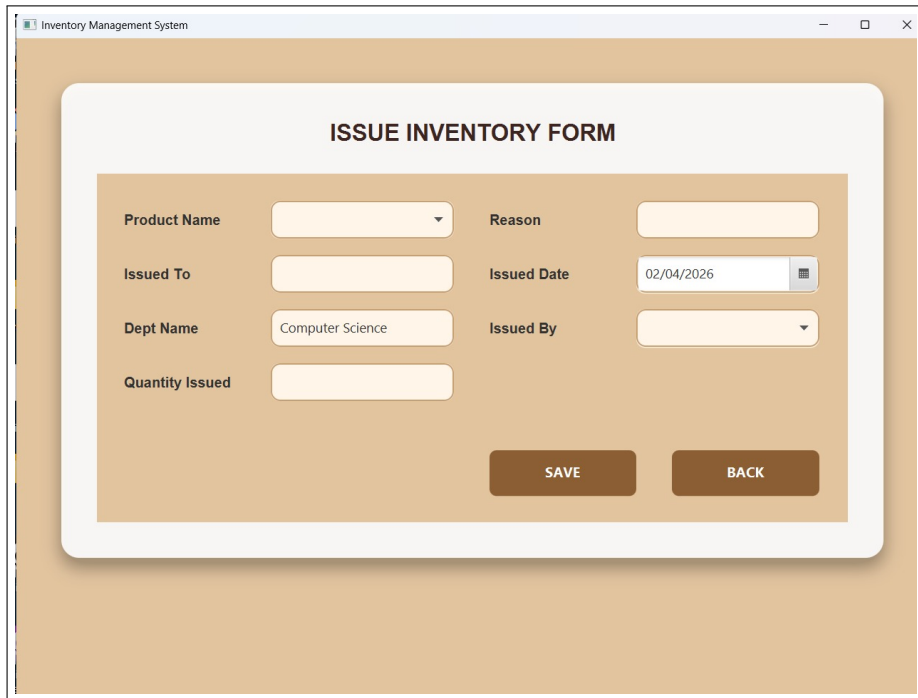
Search by name, desc

Name	Description	Min Qty	Stock	Type
A3 Paper Rim	A3 size white sheet	5	30	STN
A4 Paper Rim	A4 size white sheets	5	52	STN
Permanent Marker	black permanent marker	5	10	STN
Chairs	office chairs	5	55	FUR
Almirah	almirah	5	10	FUR
Desktop PC	RGB RAM system	5	10	CS
Keyboard Wired	wired keyboard	5	10	CS
Mouse Wired	wired mouse	5	10	CS
Candle	Candles	10	30	Misc
Map	map with stick	5	10	Misc
Handbook	liquid detail handbook	10	30	Misc
Black whiteboard marker	white board reusable markers	5	16	STN
wireless keyboard	wireless keyboard	5	20	CS
Tables	wooden tables	5	20	FUR

+ Add New Back

59
Figure 6.6: Modify — IMA

6.4 Issue Inventory Screen



The screenshot shows a web application window titled "Inventory Management System". Inside the window is a form titled "ISSUE INVENTORY FORM". The form contains the following fields and controls:

- Product Name:** A dropdown menu.
- Reason:** A text input field.
- Issued To:** A text input field.
- Issued Date:** A date picker showing "02/04/2026".
- Dept Name:** A text input field containing "Computer Science".
- Issued By:** A dropdown menu.
- Quantity Issued:** A text input field.
- Buttons:** Two buttons at the bottom right, labeled "SAVE" and "BACK".

Figure 6.7: Issue Inventory Form — IMA

6.5 Return Screen (Faculty to Inventory)

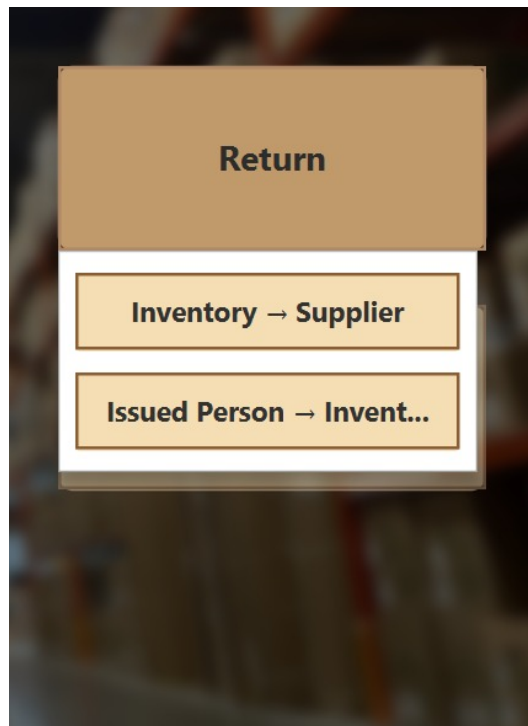


Figure 6.8: Return Button — IMA

6.6 Return to Supplier (RTS) Screen

[illegible]

6.7 View

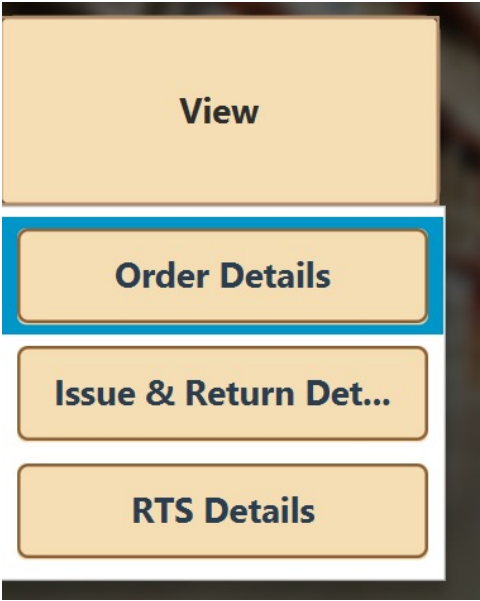
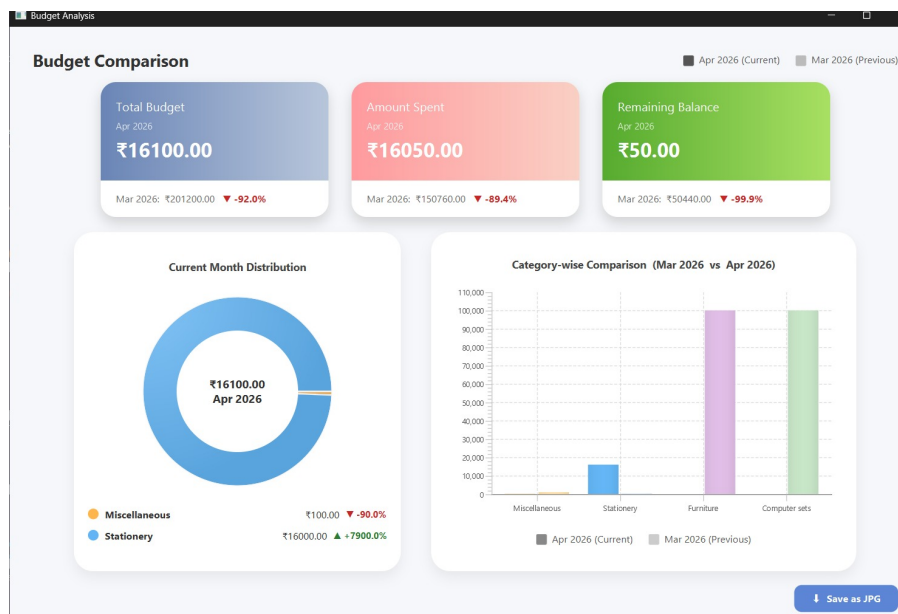


Figure 6.9: View — IMA

Order Details									
Product Name	Qty Ordered	Order Date	Order Status	Qty Received	Received Date	Bill No	Bill Amount	Bill Status	Supplier
A4 Paper	100	2026-03-31	IN_PROCESS	25	2026-03-31	B2060	Rs. 5000.00	INCOMPLETE	Stationery Hub
A4 Paper	100	2026-03-31	IN_PROCESS	25	2026-04-02	B2061	Rs. 5000.00	INCOMPLETE	Stationery Hub
A4 Paper	100	2026-03-31	IN_PROCESS	25	2026-04-02	B2062	Rs. 5000.00	INCOMPLETE	Stationery Hub
Office Chair	100	2026-03-31	IN_PROCESS	—	—	—	—	—	Furniture House
Office Chair	50	2026-03-31	PAID	25	2026-03-31	B2010	Rs. 5000.00	COMPLETE	Furniture House
Office Chair	50	2026-03-31	PAID	10	2026-03-31	B2011	Rs. 1000.00	COMPLETE	Furniture House
Office Chair	50	2026-03-31	PAID	15	2026-03-31	B2012	Rs. 20000.00	COMPLETE	Furniture House
Desktop PC	20	2026-03-25	IN_PROCESS	20	2026-03-25	B005	Rs. 700000.00	INCOMPLETE	Tech Supplies
Desktop PC	5	2025-03-12	PAID	5	2025-03-17	B5003	Rs. 150000.00	COMPLETE	Tech Supplies
Office Chair	20	2025-03-11	PAID	20	2025-03-16	B5002	Rs. 20000.00	COMPLETE	Furniture House

Figure 6.10: Order Details— IMA

6.8 Budget Analysis Screen



6.9 Reports

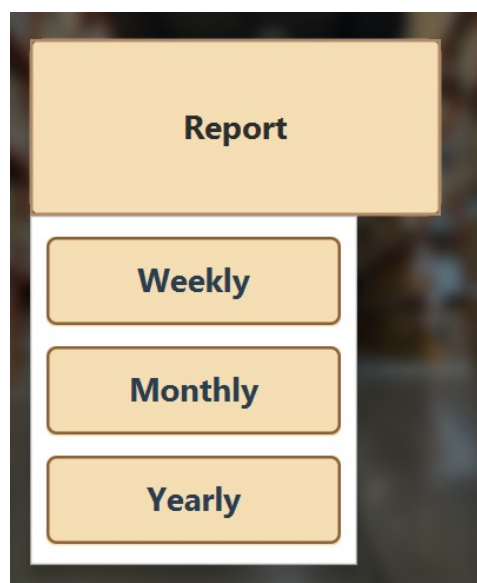


Figure 6.11: Report — IMA

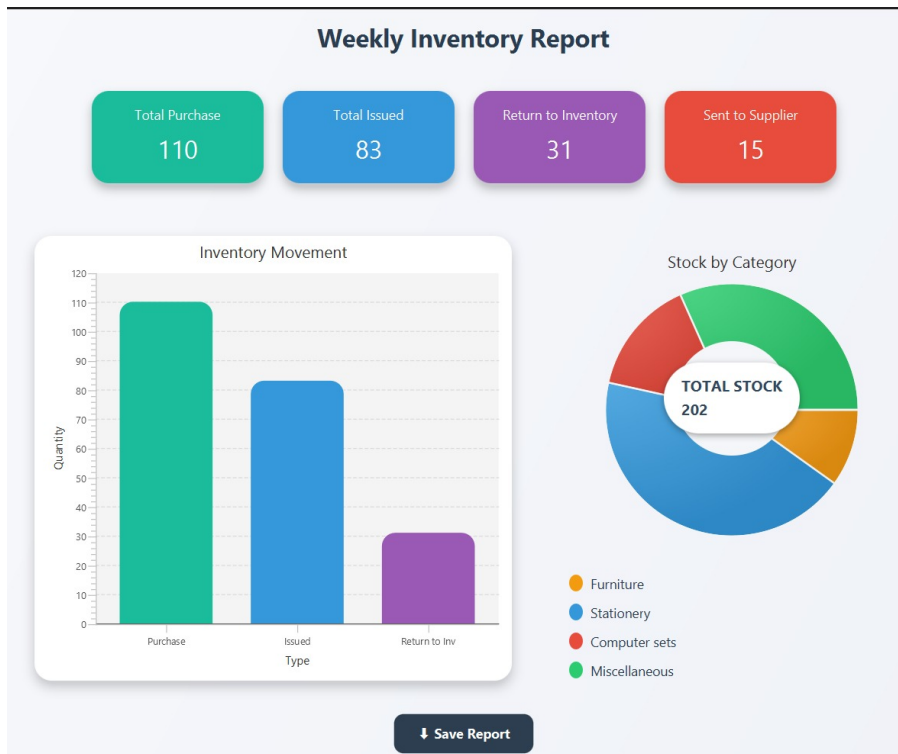
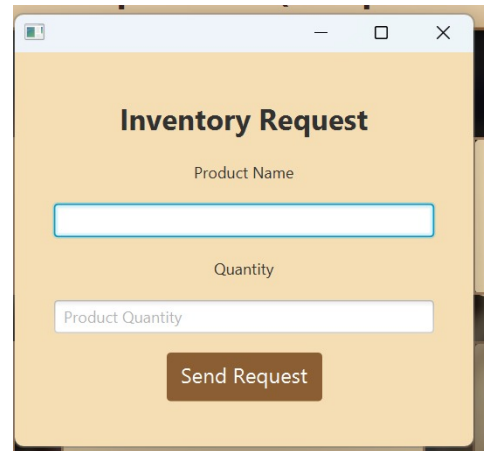


Figure 6.12: Weekly Report — IMA

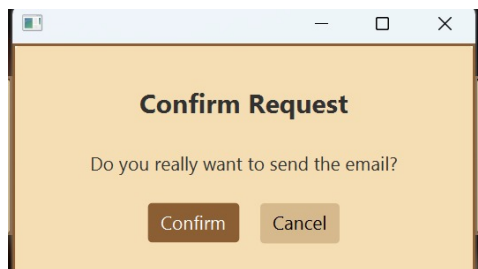
6.10 Inventory Request



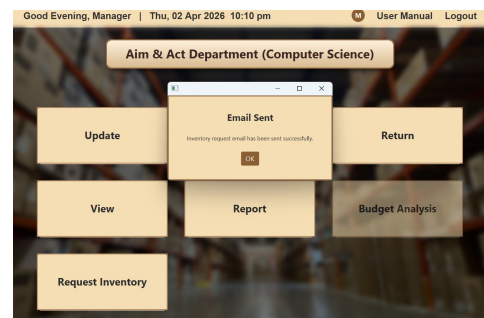
(a) Dashboard Request Option



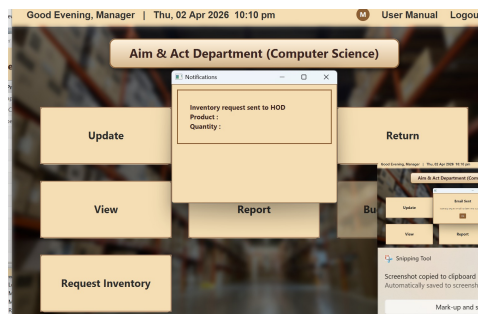
(b) Inventory Request Form



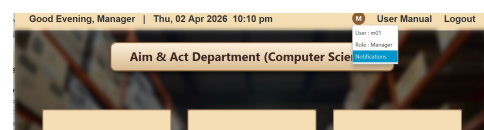
(c) Confirmation Dialog



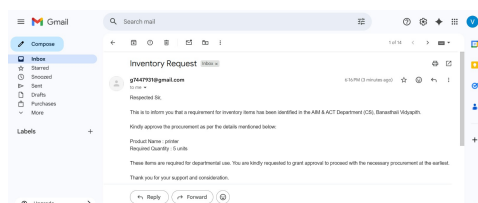
(d) Email Sent Notification



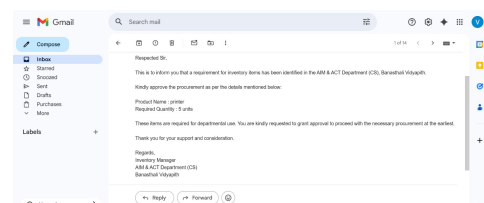
(e) System Notification Panel



(f) User Menu Notifications Access



(g) Inventory Request Email



(h) Approval Email Details

Figure 6.13: Inventory Request Workflow of the Inventory Management System

Appendix A

Glossary

Table A.1: Glossary of Terms

Term	Definition
IMA	Inventory Management Application — the desktop software developed in this project.
Inventory Manager	Office staff responsible for day-to-day inventory management and issuances.
HOD	Head of Department — administrative user with oversight and first-level approval authority.
Dean	Highest-level administrative user with final purchase approval authority.
JDBC	Java Database Connectivity — Java API for connecting to and querying relational databases.
JavaFX	Java-based GUI toolkit used for building the IMA's graphical interface, including TableView, ComboBox, and Chart components.
MySQL	Open-source relational database management system used as IMA's backend data store.
CRUD	Create, Read, Update, Delete — the four basic database operations.
DFD	Data Flow Diagram — graphical representation of data flow through the system.
ER Diagram	Entity-Relationship Diagram — illustrates database entities and their relationships.
SRS	Software Requirements Specification.
SDS	Software Design Specification.
3NF	Third Normal Form — a database normalisation standard minimising data redundancy.
RBAC	Role-Based Access Control — permissions granted based on user roles.
IN_PROCESS	Order status indicating items have been ordered but not fully received.

Term	Definition
PAID	Order status indicating the full ordered quantity has been received and billed.
COMPLETE	Bill status indicating the bill covers the full remaining order quantity.
INCOMPLETE	Bill status indicating the bill covers only a partial delivery.
ACTIVE / INACTIVE	Record status flags used for soft-delete: INACTIVE records are logically deleted but retained for audit history.
RTS	Return to Supplier — the process of returning defective or surplus items to the supplying vendor.
SENT	RTS status indicating goods have been dispatched to the supplier.
RECEIVED	RTS status indicating goods previously sent to supplier have been received back by the department.
Atomic Transaction	A database operation where all constituent SQL statements succeed together or are all rolled back on any failure, ensuring data consistency.
Soft Delete	A deletion strategy that marks records as INACTIVE rather than physically removing them, preserving audit history.
LTS	Long-Term Support — a software release designation indicating extended maintenance and security support (JAVA 25 LTS is an LTS release).

References

1. CSD026 Team (2025). *Software Requirements Specification for Inventory Management Application*, Version 1.0. Department of Computer Science, Banasthali Vidyapith, Rajasthan.
2. CSD026 Team (2025). *Software Design Specification for Inventory Management Application*, Version 1.0. Department of Computer Science, Banasthali Vidyapith, Rajasthan.
3. Patodi P., Jain S., Arora S., Jain S., Vanshree (2025). *IMA Source Code Repository*.
4. IEEE Computer Society (1998). *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998. IEEE, New York.
5. Oracle Corporation (2023). *JavaFX 17 API Documentation*. Available: <https://openjfx.io/javadoc/17/>
6. Oracle Corporation (2021). *JAVA 25 LTS Platform Specification*. Available: <https://docs.oracle.com/en/java/javase/17/>
7. Oracle Corporation (2023). *MySQL 8.0 Reference Manual*. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
8. Oracle Corporation (2022). *JDBC 4.3 API Specification*. Available: <https://docs.oracle.com/javase/tutorial/jdbc/>
9. Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional, Boston.
10. Pressman, R. S. & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education, New York.
11. Silberschatz, A., Korth, H. F. & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education, New York.
12. Sharan, K. (2015). *Learn JavaFX 17: Building User Experience and Presentation Logic in Java Applications*. Apress, New York.
13. Apache Maven Project (2024). *Maven — Introduction to the Build Lifecycle*. Available: <https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html>